UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE E TECNOLOGIE

LAUREA MAGISTRALE IN

INFORMATICA MAGISTRALE (LM-18)

PATIENT KNOWLEDGE GRAPH REPRESENTATION USING NON-EUCLIDEAN METRIC

Relatore:

Dott. Mauricio Abel Soto Gomez

Correlatrice:

Dott.ssa Elena Casiraghi

Studente:

Andrea Moretti

ID: 47263A

Anno accademico: 2024/2025

ABSTRACT

La ricerca esplora l'applicazione di tecniche di Graph Embedding per la rappresentazione di Knowledge Graph (KG) in ambito medico, concentrandosi in particolare sul supporto alla diagnosi delle malattie mendeliane. Poiché i dati biomedici e le reti complesse presentano spesso una topologia fortemente gerarchica, i modelli di rappresentazione tradizionali basati sullo spazio euclideo faticano a modellarli in modo efficiente, causando distorsioni e perdita di informazioni.

Per analizzare le possibili soluzioni a questa problematica è stato fatto uno studio diviso in tre principali passi sperimentali:

- Geometria Euclidea vs. Iperbolica: la ricerca confronta le prestazioni delle Graph Convolutional Networks tradizionali (GCN) con le loro controparti iperboliche (HGCN). Gli esperimenti dimostrano che lo spazio iperbolico, grazie alla sua capacità di espandersi esponenzialmente, modella molto meglio le gerarchie complesse. Nel task di link prediction sulle malattie mendeliane, il modello HGCN ha ottenuto un miglioramento medio del 23% nella metrica ROC rispetto a GCN, compensando ampiamente i tempi di calcolo più lunghi.
- Modelli Omogenei vs. Eterogenei: I modelli di tipo Graph Convolutional operano principalmente su grafi omogenei, perdendo l'informazione contenuta nei diversi tipi di nodi e archi tipica dei Knowledge Graph. Confrontando i modelli omogenei (GCN/HGCN) con modelli progettati per grafi eterogenei (AttE/AttH), i risultati evidenziano un salto qualitativo decisivo. L'utilizzo di modelli su grafi eterogenei ha portato a un drastico aumento della precisione predittiva, generando la riduzione più significativa del gap di performance, pari al 71,9%.
- Coerenza dei dati biomedici: lo studio dimostra quanto sia cruciale la corretta contestualizzazione dei dati inseriti nel grafo. Un test sull'inclusione di nodi di tipo *Gene* senza l'affiancamento dei nodi *Protein* ha causato un peggioramento delle prestazioni, confermando che l'aggiunta di informazioni biologiche frammentate o non strutturate in modo coerente complica lo spazio di apprendimento senza fornire reali vantaggi predittivi.

Contents

1 Introduction **1**

2 Definizioni e metodi **4**

2.1 Graph embedding 4

2.2 Tecniche di graph embedding 4

2.3 Hyperbolic Embedding e Poincaré model 9

2.4 Graph Convolutional Network 13

2.5 Knowledge Graph 16

2.6 Modelli eterogenei per Knowledge Graph embedding 17

3 Sperimentazioni **19**

3.1 Schema metodologico 19

3.2 Librerie e dataset 20

3.2.1 Knowledge Graph per malattie mendeliane 22

3.3 Metriche 26

3.4 Studio preliminare delle HGCN su dataset di letteratura medica 28

3.5 Confronto pratico HGCN vs GCN su dataset di letteratura medica 29

3.6 Caso studio: knowledge graph per malattie mendeliane 33

3.6.1 Motivazioni della sperimentazione 33

3.6.2 Applicazione dei modelli GCN/HGCN sul KG delle malattie mendeliane 34

3.6.3 Training e risultati 36

3.6.4 Applicazione e analisi di metodi per grafi eterogenei 40

3.6.5 Comparazione diretta tra metodi omogenei e eterogenei 43

3.6.6 Analisi finale sul KG ridotto: inclusione dei nodi di tipo *Gene* 45

4 Conclusioni **49**

4.1 Sviluppi futuri e possibili migliorie 50

5 Bibliography **51**

6 Ringraziamenti **53**

1 INTRODUCTION

Il campo della medicina moderna sta attraversando una fase di profonda trasformazione, spinta dalla disponibilità di dati molecolari e clinici senza precedenti. Tuttavia, nonostante i progressi nel sequenziamento genomico e nella digitalizzazione delle informazioni sanitarie, la medicina di precisione si scontra ancora con importanti sfide, in particolare nell'ambito delle patologie ereditarie.

Secondo il National Institutes of Health (NIH), si stima che esistano tra le 7 000 e le 10 000 malattie mendeliane distinte, che colpiscono globalmente circa 2 milioni di persone. Per molti di questi pazienti, l'iter diagnostico rappresenta un percorso che può durare anni e che spesso non porta a una risposta definitiva. Nonostante l'applicazione di tecniche avanzate come il sequenziamento dell'esoma e del genoma (WES e WGS), il tasso di successo diagnostico rimane limitato: oltre il 50% dei pazienti con sospetta patologia mendeliana non riceve una diagnosi molecolare certa [1].

Le malattie mendeliane, causate da mutazioni in un singolo gene, rappresentano un problema critico in questo contesto. Sebbene la causa genetica sia chiara a livello teorico, la manifestazione fenotipica, ovvero l'insieme dei sintomi e delle caratteristiche osservabili, è spesso estremamente eterogenea. Varianti nello stesso gene possono causare malattie diverse, così come geni diversi possono manifestarsi con sintomi quasi identici. Questa complessità non è solo biologica, ma anche informativa: la conoscenza medica necessaria per interpretare questi dati è frammentata in migliaia di pubblicazioni scientifiche, database genetici e cartelle cliniche non strutturate.

Il problema fondamentale della diagnosi assistita dal Machine Learning risiede nella natura stessa di queste informazioni. I dati medici non sono entità isolate, ma formano una rete intrinseca di relazioni: un sintomo (fenotipo) è collegato a una malattia, la quale è associata a uno o più geni, che a loro volta interagiscono in complessi processi biologici e vie metaboliche. Tradurre questa rete di interdipendenze in un formato che un algoritmo di intelligenza artificiale possa apprendere è una sfida centrale della bioinformatica contemporanea.

Storicamente, l'approccio computazionale alla medicina si è basato su modelli tabulari o gerarchie rigide. Tuttavia, data la sua dinamicità e complessità, la biologia umana non è schematizzabile in una semplice tabella. Molte delle strutture che definiscono la nostra conoscenza medica, come la Human Phenotype Ontology (HPO) o la Gene Ontology (GO), sono organizzate in grafi diretti aciclici che presentano una forte natura gerarchica. In queste strutture, i concetti si ramificano dal generale al più specifico con una densità che cresce in modo esponenziale man mano che si scende nei livelli di dettaglio.

È proprio in questo contesto che emerge la necessità di strumenti matematici e informatici più sofisticati. I modelli tradizionali di Machine Learning operano tipicamente in spazi Euclidei, che però falliscono nel rappresentare fedelmente strutture che crescono esponenzialmente. Quando si cerca di mappare una complessa gerarchia medica in uno spazio piatto (Euclideo), perdiamo inevitabilmente informazioni cruciali sulla vicinanza e sull'importanza relativa dei nodi, un limite che può compromettere la precisione di una diagnosi automatizzata.

Questa ricerca affronta direttamente la problematica della diagnosi automatizzata delle malattie mendeliane, proponendo l'utilizzo di geometrie non euclidee per la rappresentazione della conoscenza del paziente. L'idea principale è che, per modellare correttamente la complessità medica, lo "spazio" in cui i dati vengono analizzati debba riflettere la struttura intrinseca dei dati stessi. In particolare, la geometria iperbolica offre una proprietà unica: la sua capacità di espandersi esponenzialmente la rende l'ambiente naturale per ospitare grafi gerarchici e reti biologiche complesse, minimizzando la distorsione informativa.

L'obiettivo di questo lavoro è quindi indagare come la rappresentazione dei pazienti all'interno di un Knowledge Graph, processata attraverso tecniche di Graph Embedding in spazi iperbolici, possa migliorare la capacità di prevedere associazioni tra profili clinici dei pazienti e malattie. Invece di limitarsi a un'analisi isolata del DNA, l'approccio qui proposto mira a sfruttare l'intera rete di relazioni bio-mediche per supportare il processo decisionale.

Lo studio prova che l'uso di geometrie e metriche avanzate nell'apprendimento su grafi è fondamentale per catturare la complessità biologica e ottimizzare l'accuratezza diagnostica nelle malattie rare.

Di seguito, in breve, un elenco dei passi sperimentali e una lista di risultati:

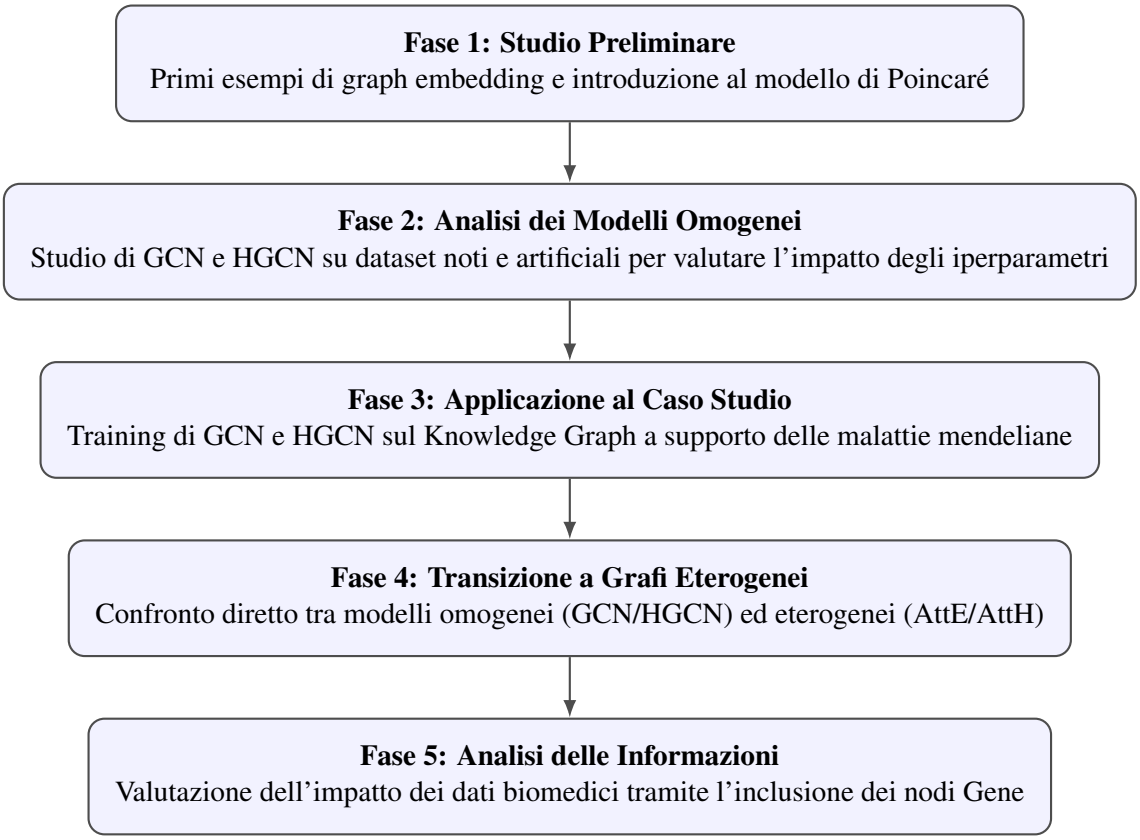


Figure 1: Roadmap metodologica delle fasi sperimentali

Risultati principali in breve

+23% **Vantaggio Iperbolico:** Il modello HGCN ha superato costantemente la controparte euclidea nella predizione di nuovi legami malattia-fenotipo, confermando l'efficienza della geometria non-euclidea.

92% **Precisione (Hits@1):** L'adozione di modelli per grafi eterogenei (AttH) ha permesso di raggiungere alta precisione nella diagnosi corretta al primo tentativo, riducendo l'errore del 71,9% rispetto a HGCN.

Insight **Coerenza Biologica:** La sperimentazione ha dimostrato che la sola aggiunta di dati genomici (nodi Gene) senza il contesto proteico peggiora le performance, evidenziando l'importanza delle relazioni funzionali nel Knowledge Graph.

2 DEFINIZIONI E METODI

2.1 Graph embedding

Il graph embedding, è una tecnica di machine learning che consente di rappresentare i nodi di un grafo attraverso un vettore di dimensione ridotta, preservando il più possibile le proprietà strutturali e semantiche del grafo originale. Questi vettori possono essere utilizzati come input per modelli di machine learning per supportare diversi task di predizione, come la classificazione dei nodi, il link prediction o il clustering.

L'adozione di rappresentazioni dei nodi arricchite, che integrano informazioni locali e pattern globali, consente di incrementare le performance nei modelli predittivi rispetto alle tecniche di apprendimento standard.

2.2 Tecniche di graph embedding

In letteratura esistono diversi approcci per implementare il graph embedding, tra cui metodi basati su fattorizzazione di matrici (come Laplacian Eigenmaps), modelli basati su reti neurali (ad esempio Graph Neural Networks) e tecniche basate su campionamento del grafo.

Tra queste, i metodi basati su *random walk* hanno ottenuto particolare successo grazie alla loro capacità di catturare sia relazioni locali sia proprietà più globali della struttura del grafo. L'idea alla base è quella di esplorare il grafo generando sequenze di nodi tramite cammini casuali, in modo da trasformare l'informazione strutturale in un formato sequenziale (ad esempio come vettore).

La modalità con la quale viene fatta l'esplorazione del grafo incide nella rappresentazione vettoriale, per questo è importante scegliere coerentemente con l'obiettivo finale i parametri del random walk.

DeepWalk [2] è uno dei primi algoritmi di graph embedding basati su random walk. L'idea principale di DeepWalk consiste nel trasformare il grafo in un insieme di sequenze di nodi, analoghe a frasi nel linguaggio naturale. Per ogni nodo del grafo vengono generati diversi cammini casuali (random walk) di lunghezza prefissata, in cui ogni step consiste nel passaggio a un nodo adiacente scelto in modo uniforme.

Le sequenze (cammini) ottenute vengono poi utilizzate come input per un modello di apprendimento non supervisionato che ha l'obiettivo di apprendere rappresentazioni vettoriali dei nodi a partire dalle loro occorrenze nelle sequenze generate. Ogni nodo viene quindi raggruppato insieme ai nodi che compaiono nel suo intorno all'interno di una finestra di ampiezza fissata, i quali ne definiscono il contesto.

Per l'apprendimento delle rappresentazioni dei nodi DeepWalk utilizza skip-gram [3], un noto modello addestrato a massimizzare la probabilità di osservare i nodi di contesto dato un nodo centrale. In questo modo, nodi che compaiono frequentemente negli stessi contesti (ovvero che sono strutturalmente vicini o connessi nel grafo) vengono rappresentati da vettori simili nello spazio latente, riflettendo la loro vicinanza strutturale nel grafo.

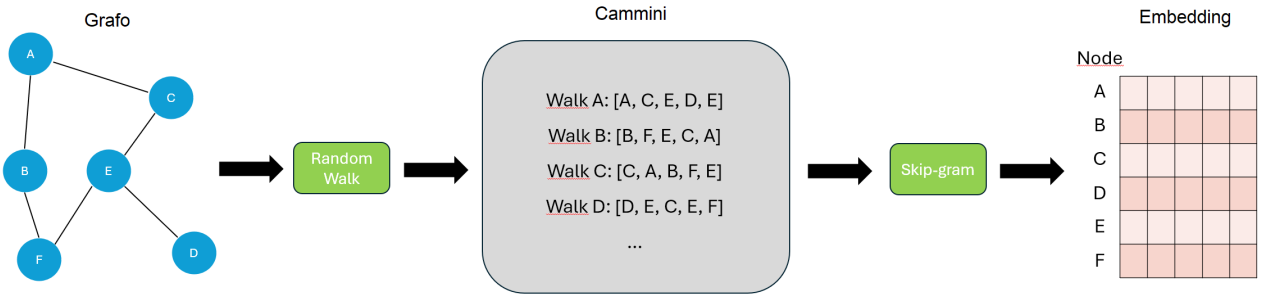


Figure 2: Raffigurazione ad alto livello del funzionamento di DeepWalk e Node2Vec

DeepWalk effettua cammini casuali uniformi, in cui la probabilità di transizione verso ciascun nodo adiacente è la stessa (assenza di bias). Questa strategia, sebbene semplice ed efficace, limita la capacità del modello di controllare il tipo di esplorazione del grafo.

Un altro metodo noto in letteratura, che agisce in maniera molto simile a DeepWalk è Node2Vec [4], un algoritmo che genera embedding dei nodi attraverso l'esplorazione del grafo con cammini casuali (random walk), con l'unica differenza che per esplorare il grafo effettua random walk asimmetrici.

Il random walk asimmetrico, a differenza di quello standard, dirige la sua esplorazione attaverso 2 principali parametri:

- **p**: return parameter, controlla la probabilità di tornare indietro al nodo precedente visitato;
- **q**: in-out parameter, controlla la probabilità di esplorare localmente o globalmente; un valore basso predilige un'esplorazione simile a una BFS, mentre un valore alto dirige l'esplorazione verso a una DFS.

L'uso di random walk asimmetrici permette a Node2Vec di modellare l'esplorazione in modo più mirato, producendo rappresentazioni dei nodi fortemente dipendenti dal contesto strutturale locale.

Generare rappresentazioni dei nodi sfruttando i random walks però porta il modello a non considerare il tipo o il peso degli archi: considera solamente la presenza di un arco tra i nodi. Per questo motivo, in contesti in cui gli archi hanno semantica diversa (es. in un grafo eterogeneo o multigrafo), Node2Vec non è in grado di distinguere le differenze tra archi, comportando una limitazione importante se l'informazione descritta dagli archi è rilevante per il compito (ad esempio classificazione, predizione, ecc.).

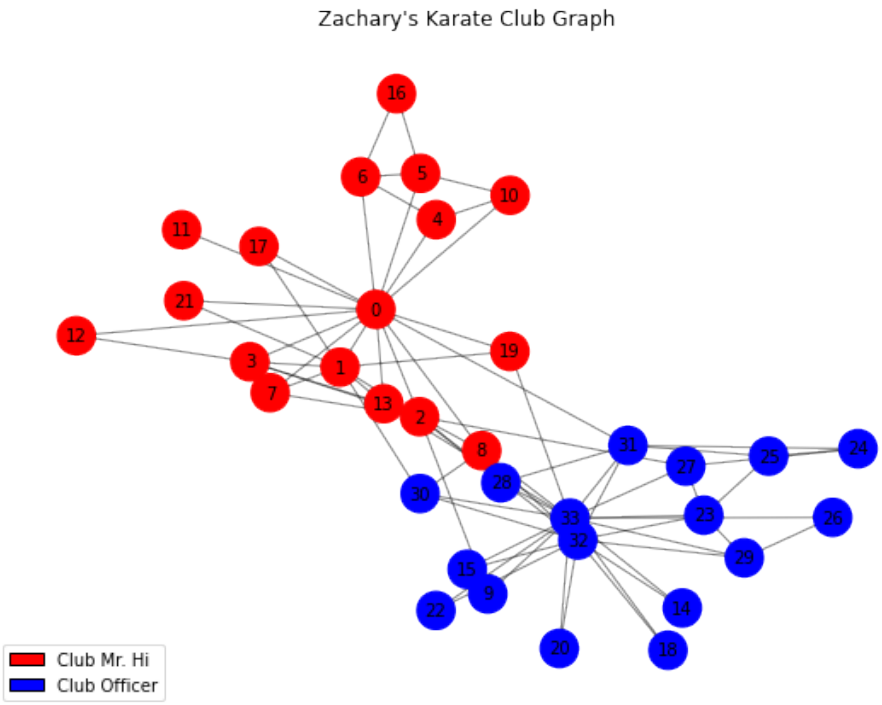


Figure 3: Grafo di esempio utilizzato

Tra i diversi possibili esempi applicativi, in questa sezione viene presentato un caso concreto che mostra come sia possibile ottenere rappresentazioni vettoriali dei nodi di un grafo (Figura 3) utilizzando l'algoritmo Node2Vec.

L'embedding del grafo è stato ottenuto associando 50 cammini di lunghezza 20 per ogni nodo, fissando i parametri per ottenere un'esplorazione moderatamente ampia e bilanciata (p=2 e q=1).

Successivamente, per permettere la visualizzazione in un piano bidimensionale, l'embedding è stato dato in input a un modello specializzato in riduzione della dimensionalità. In questo esempio è stata applicata la tecnica di t-distributed Stochastic Neighbor Embedding (t-SNE), particolarmente efficace per dataset complessi, fissando come dimensione finale 2; infine il risultato è stato proiettato in un grafico (Figura 4).

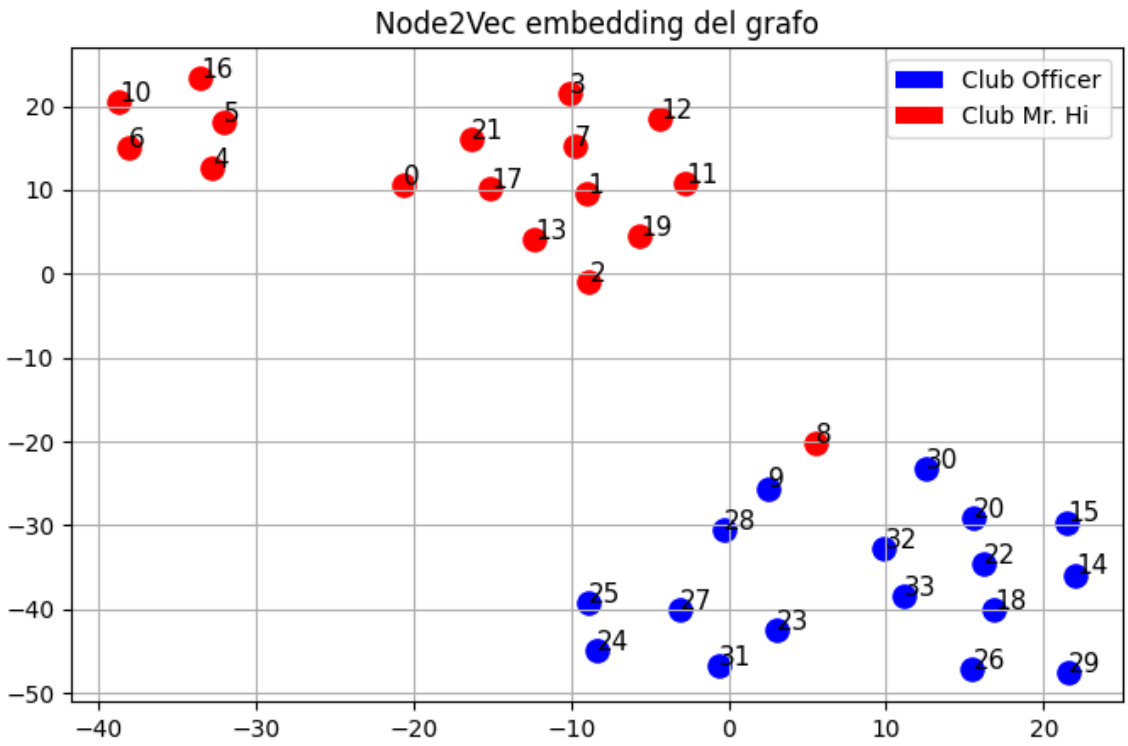


Figure 4: Proiezione dell'embedding ottenuto dall'applicazione di Node2Vec e successivamente ridotto a 2 dimensioni con t-SNE

Come mostrato in Figura 4, la rappresentazione finale ottenuta tramite Node2Vec preserva in maniera efficace la struttura del grafo originale. In particolare, si osserva un chiaro raggruppamento del nucleo di nodi [4, 5, 6, 10, 16], che risultano vicini nello spazio embedding, a conferma della coesione dei loro collegamenti nel grafo. Inoltre, il nodo 8, pur appartenendo al Club Mr. Hi, mantiene numerose connessioni con nodi del Club Officer; di conseguenza, il modello lo ha posizionato in una posizione intermedia, più vicino ai nodi del Club Officer, riflettendo correttamente la sua natura di ponte tra i due gruppi.

2.3 Hyperbolic Embedding e Poincaré model

I modelli menzionati finora costruiscono le rappresentazioni dei grafi assumendo uno spazio latente euclideo. Tuttavia, questa scelta introduce limitazioni rilevanti. In particolare, gli embedding euclidei (come quelli prodotti da Node2Vec) faticano a rappresentare in modo efficiente grafi caratterizzati da strutture fortemente gerarchiche.

Molti dataset reali (reti complesse, grafi di conoscenza, ontologie), soprattutto in ambito biomedico, presentano una topologia gerarchica ad albero (tree-like). In un albero con fattore di ramificazione b, il numero di nodi a profondità h cresce in modo esponenziale: $O(b^h)$.

Nello spazio euclideo $\mathbb{R}^d$ il volume V di una sfera di raggio r cresce solo in modo polinomiale rispetto al raggio: $V \propto r^d$. La rappresentazione di dati che crescono esponenzialmente (grafi gerarchici) in uno spazio che cresce polinomialmente (euclideo) causa una distorsione. I nodi foglia vengono "schiacciati" ai bordi, perdendo le informazioni sulle loro distanze relative (il cosiddetto *crowding problem*).

Per superare queste limitazioni è necessario introdurre la geometria iperbolica. Lo spazio iperbolico è un tipo di geometria non euclidea caratterizzata da una curvatura costante e negativa. Una proprietà fondamentale dello spazio iperbolico è che il volume V di una sfera di raggio r cresce in modo esponenziale rispetto al raggio: $V \propto e^r$.

Questa crescita esponenziale dello spazio disponibile corrisponde perfettamente alla crescita esponenziale dei nodi in un grafo gerarchico ad albero. Di conseguenza, le gerarchie possono essere incorporate nello spazio iperbolico con una distorsione molto bassa, assegnando i nodi radice vicino all'origine e le foglie verso il bordo dello spazio. Questa capacità risolve efficacemente il crowding problem, preservando le distanze e la struttura intrinseca dei grafi complessi.

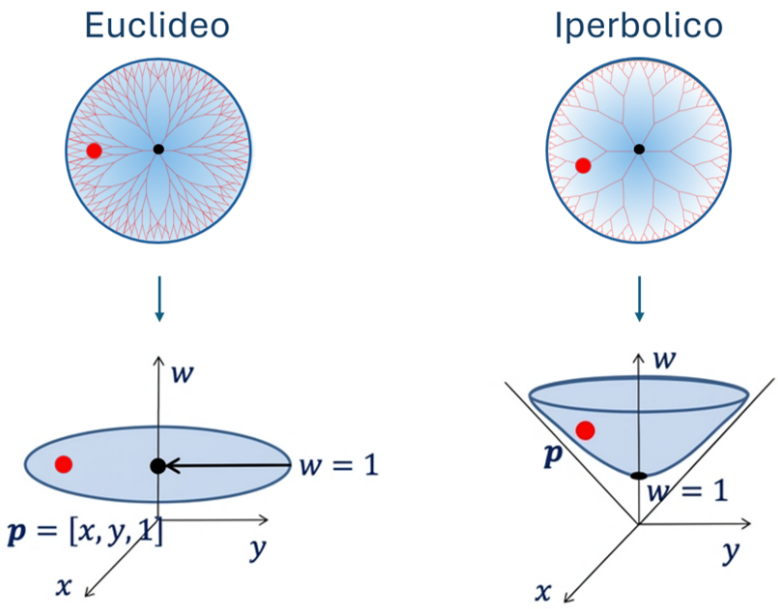


Figure 5: Visualizzazione geometrica della rappresentazione di un punto p nello spazio Euclideo (sinistra) e nello spazio Iperbolico (destra). Nel caso Euclideo, il punto giace su un piano a quota costante $w = 1$. Nel modello iperbolico (modello di Lorentz), il punto è proiettato sulla falda superiore di un iperboloide a due fogli, dove la coordinata w varia in funzione della distanza dall'origine per preservare la curvatura negativa dello spazio. Nota come nel disco iperbolico (in alto a destra) le ramificazioni diventino più fitte verso il bordo. (Fonte [5])

Il passaggio logico fondamentale per superare i limiti del graph embedding classico basato sullo spazio euclideo consiste nel rappresentare il grafo sulla base della geometria iperbolica. Invece di proiettare i nodi su un piano "piatto", gli embedding iperbolici utilizzano una geometria a curvatura negativa costante, dove le proprietà metriche cambiano radicalmente. Questo tipo di rappresentazione è definito Hyperbolic Embedding [6].

Esistono diversi modelli matematici per visualizzare e calcolare queste coordinate, ma i due più utilizzati nel Machine Learning su grafi sono il Disco di Poincaré, il Modello dell'Iperboloide (Lorentz) e il Product of Balls, una combinazione di spazi curvi.

Il modello di Poincaré è il più intuitivo per visualizzare come lo spazio si "espande" verso il bordo (Figura 6). Il modello è definito come un disco unitario n-dimensionale $\mathbb{B}^n = \{x \in \mathbb{R}^n : \|x\| < 1\}$, ovvero una sfera n-dimensionale con raggio pari a 1.

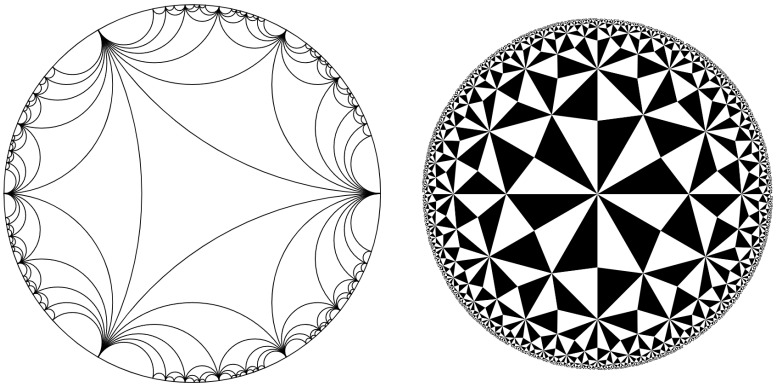


Figure 6: Rappresentazione grafica dell'espansione del Poincaré Disk (Wikipedia)

La distanza tra due punti non è una linea retta euclidea, ma cresce infinitamente man mano che ci si avvicina al bordo del disco (il cerchio unitario in Figura 6). La radice viene posizionata vicino all'origine $(0,0)$, mentre i figli e le foglie vengono spinti verso il bordo. Poiché la distanza iperbolica diverge verso il bordo, c'è "infinito spazio" per separare i miliardi di foglie di una gerarchia complessa senza schiacciarle.

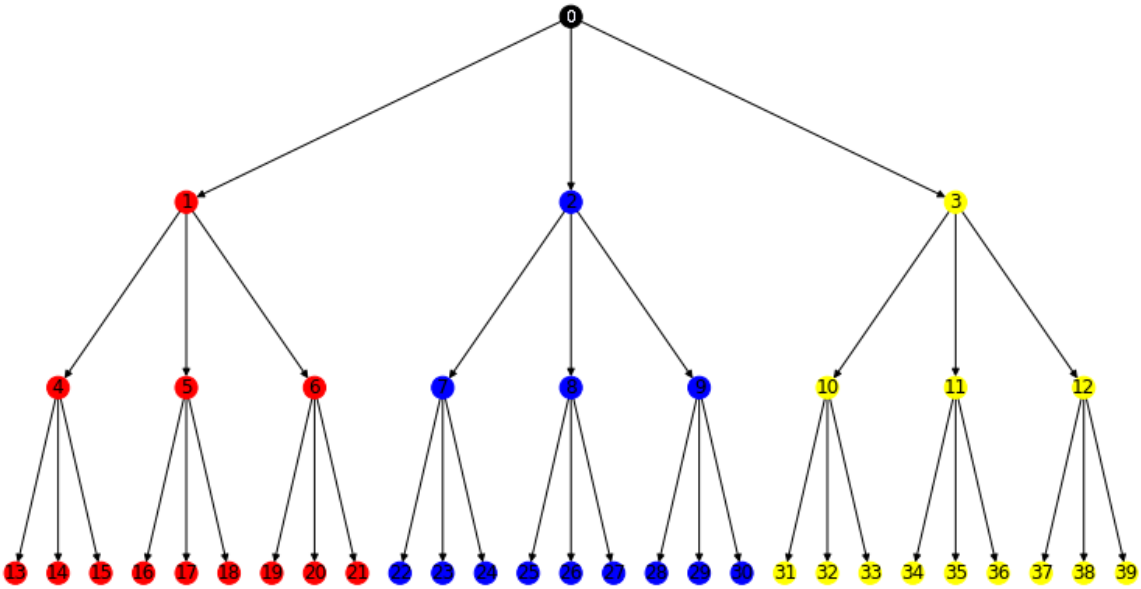


Figure 7: Grafo gerarchico di esempio

Per ottenere l'embeddig iperbolico di un grafo (ad esempio Figura 7) tipicamente si costruisce una piccola rete neurale e si utilizza lo strato hidden per ricavare la rappresentazione vettoriale dei nodi.

Poiché le operazioni matematiche negli spazi iperbolici differiscono da quelle negli spazi euclidei, è necessario adottare strumenti e operazioni specifiche per rappresentare e manipolare correttamente i dati nel modello del disco di Poincaré.

Solitamente si definiscono dei *manifold*, strutture matematiche dotate di operazioni che permettono di mantenere la coerenza dei dati rispetto alla geometria dello spazio in cui sono rappresentati. Nel modello di Poincaré spesso si utilizza invece una mappatura tra spazio euclideo e spazio iperbolico, tipicamente tramite la mappatura esponenziale e logaritmica, che permette di trasferire i dati nel cosiddetto spazio tangente, applicare le operazioni euclidee e poi riportare il risultato nello spazio iperbolico.

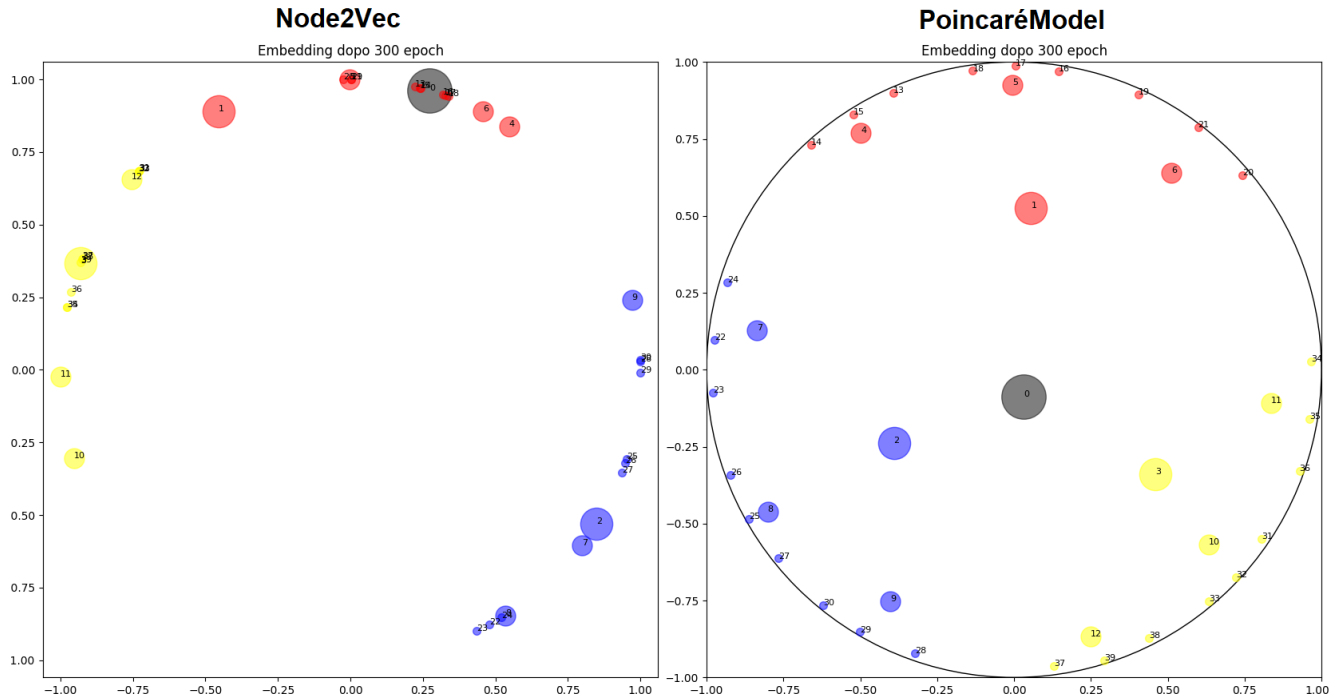


Figure 8: Confronto di rappresentazione del grafo gerarchico ottenuta tramite Node2Vec vs metodo PoincareModel di Gensim

L'esempio di rappresentazione iperbolica mostrato in Figura 8 (destra) è stato ottenuto utilizzando il metodo PoincareModel fornito da Gensim, con un fattore di *negative sampling* pari a 5, che specifica il numero di coppie di nodi non connessi da considerare nella valutazione della loss per ogni nodo vicinio confrontato, e come grandezza della rappresentazione 2, in modo che possa essere proiettata direttamente all'interno del Poincaré Disk. Per l'embedding euclideo in Figura 8 (sinistra) è stato utilizzato Node2Vec fissando i parametri del random walk p=1 e q=0.5, mantenendo la rappresentazione a 2.

2.4 Graph Convolutional Network

Negli esempi precedenti sono state mostrate tecniche euclidee e iperboliche per ottenere l'embedding di un grafo a partire esclusivamente dalla sua struttura. In un contesto medico numerose informazioni possono essere rappresentate attraverso un grafo, come ad esempio interazioni proteina-proteina o strutture molecolari. L'informazione, quindi, non risiede solamente nelle caratteristiche dei singoli elementi (nodi), ma emerge in modo determinante dalle relazioni (archi) che li connettono.

Per integrare efficacemente sia la struttura del grafo sia l'informazione semantica associata ai nodi, sono stati sviluppati modelli di apprendimento più espressivi, tra cui le Graph Convolutional Networks (GCN). L'obiettivo fondamentale di una GCN è apprendere una rappresentazione vettoriale (embedding) per ogni nodo che sia in grado di codificare simultaneamente due tipi di informazioni:

- Le feature locali: le caratteristiche intrinseche del nodo stesso, ovvero tutte le informazioni dei nodi vicini.
- La struttura topologica: il contesto fornito dalla posizione del nodo e i vicini, rapportato all'interno dell'intero grafo.

La caratteristica principale delle GCN risiede nel meccanismo di aggregazione dei messaggi: attraverso successivi livelli di propagazione, ogni nodo raccoglie e trasforma le informazioni provenienti dai suoi vicini, raffinando progressivamente la propria rappresentazione latente. Questo approccio permette alla rete di catturare dipendenze strutturali a lungo raggio, abilitando task complessi come la classificazione dei nodi (node classification) e la previsione di nuovi archi (link prediction).

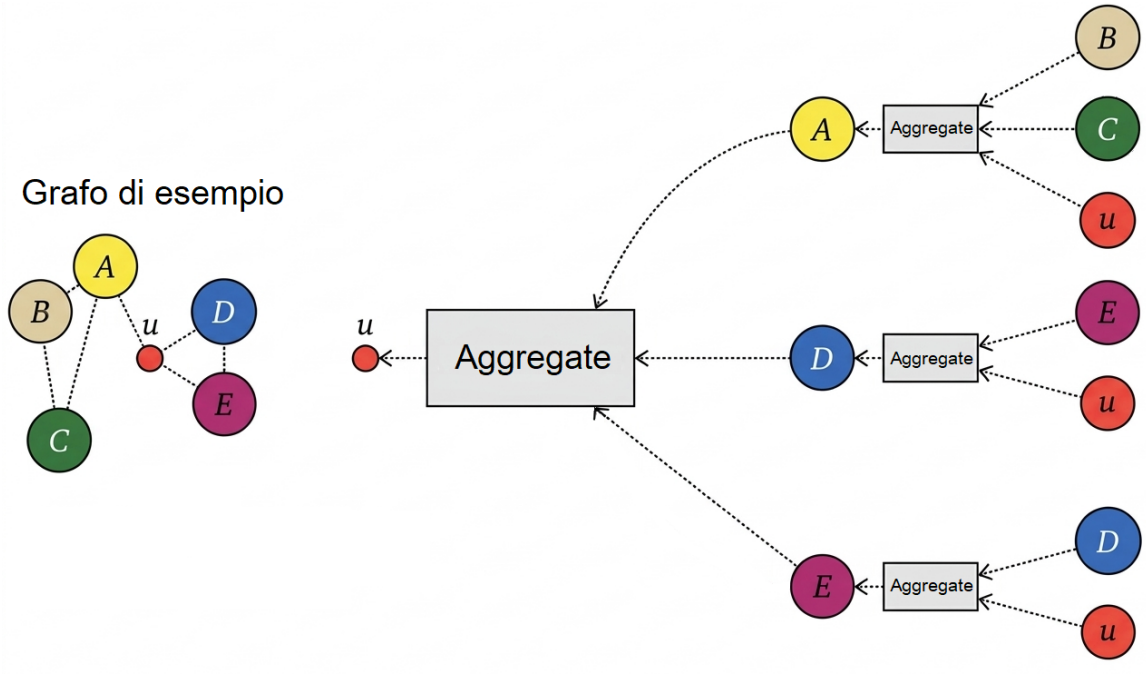


Figure 9: Schema di esempio dell'operazione di aggregazione di una GCN

Durante la ricerca i modelli GCN utilizzati sono stati allenati esclusivamente sul task di link prediction. Questo tipo di task all'interno dei modelli è implementato tramite una funzione di score. Lo score di un potenziale arco tra i nodi u e v è spesso calcolato basandosi sulla distanza all'interno dello spazio di rappresentazione $d_{\mathbb{B}}(z_u, z_v)$ ($\mathbb{B}$ lo spazio di embedding) nei rispettivi embedding e rappresenta la probabilità dell'esistenza di una connessione tra u e v.

Lo score s_{uv} viene solitamente definito tramite una funzione sigmoide:

$$s_{uv} = \sigma(-d_{\mathbb{B}}(z_u, z_v)) \tag{1}$$

dove z_u, z_v rappresentano l'embedding dei nodi u e v.

Le Hyperbolic Graph Convolutional Networks (HGCN) [7] rappresentano un'estensione delle tradizionali GCN al dominio iperbolico: invece di operare nello spazio euclideo, esse costruiscono rappresentazioni in spazi con curvatura negativa, più adatte a modellare dati con una complessa struttura gerarchica profonda.

In queste reti ogni operazione di convoluzione e aggregazione dei nodi è definita direttamente nello spazio iperbolico, sfruttando mappe di proiezione (logaritmiche ed esponenziali) che permettono di passare coerentemente tra spazi tangenti ed iperbolici. Questo approccio consente alle HGCN di catturare in modo più efficace relazioni complesse e multilivello, riducendo la distorsione geometrica e ottenendo embedding più compatti rispetto ai modelli euclidei.

In contesti come quello medico, dove i grafi sono spesso caratterizzati da strutture ontologiche e gerarchie latenti, le HGCN permettono di integrare struttura e informazioni, ottenendo rappresentazioni significativamente più espressive.

Va specificato però che questa tipologia di reti opera esclusivamente su grafi omogenei, ovvero grafi in cui tutti gli archi condividono la stessa semantica. Questo differisce dai Knowledge Graph, che invece sono per definizione eterogenei, poiché includono diversi tipi di entità e relazioni semanticamente distinte, ciascuna potenzialmente con un proprio ruolo.

Questo implica che le HGCN risultano particolarmente efficaci in contesti in cui i dati possono essere rappresentati tramite una struttura gerarchica o ad albero, ma non sono progettate per gestire direttamente l'eterogeneità relazionale tipica dei Knowledge Graph. Per questo motivo quando entrano in gioco più tipi di relazione la classica formulazione delle HGCN spesso non è sufficiente.

2.5 Knowledge Graph

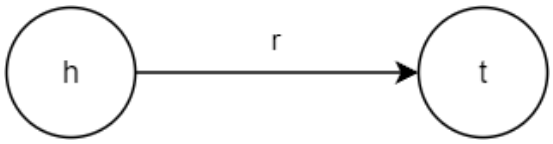
Negli ultimi decenni, l'avanzamento tecnologico nel settore biomedico ha determinato una crescita esponenziale nella produzione e raccolta di dati clinici, genomici e sperimentali. Strumenti come cartelle cliniche elettroniche, dispositivi indossabili o tecnologie di sequenziamento avanzati contribuiscono quotidianamente alla generazione di informazioni eterogenee e ad alta complessità. Tuttavia, questa abbondanza di dati spesso risulta frammentata e complessa da integrare, limitando la possibilità di estrarre conoscenza utile per la ricerca e la diagnosi di malattie.

Per risolvere questa frammentazione, la ricerca si è spostata verso la rappresentazione della conoscenza (Knowledge Representation). L'obiettivo non è più solo memorizzare "stringhe" di testo, ma identificare "entità" concettuali e le loro interdipendenze semantiche.

Soprattutto in medicina, l'informazione non vive isolata: un gene interagisce con una proteina, che è il target di un farmaco, il quale può causare un effetto collaterale in un paziente con una specifica mutazione genetica.

Per rappresentare relazioni strutturali eterogenee, una delle soluzioni più efficaci è data dai Knowledge Graph (KG): essi costituiscono un modello basato sulla teoria dei grafi e sulla rappresentazione della conoscenza, offrendo un modello flessibile in cui le informazioni sono rappresentate come una rete dinamica di relazioni. A differenza di un database relazionale classico, il KG eccelle nel rappresentare dati semi-strutturati e relazioni complesse.

Formalmente, la conoscenza è espressa tramite triple nella forma:



$(soggetto, predicato, oggetto)$ formalmente: (h, r, t)

dove h è l'entità di testa (head), r è la relazione e t è l'entità di coda (tail).

2.6 Modelli eterogenei per Knowledge Graph embedding

La rappresentazione accurata di Knowledge Graph risulta molto complessa, infatti, i modelli di HGCN e GCN operano solo sul campo omogeneo, non considerando di conseguenza il tipo degli archi e dei nodi. Questa limitazione non permette quindi di rappresentare in maniera efficiente le informazioni contenute nel grafo, ma solamente la struttura e le connessioni che derivano da essa.

Per rappresentare in modo completo un KG, sfruttando tutte le informazioni contenute al suo interno è necessario adottare dei modelli di embedding più complessi.

L'obiettivo fondamentale dei Knowledge Graph Embedding (KGE) risiede nella proiezione degli elementi discreti di un KG (nodi $v \in V$ e relazioni $r \in R$) in uno spazio geometrico latente U, tipicamente identificato con lo spazio euclideo $\mathbb{R}^d$.

La scelta dello spazio U e della sua dimensione d definisce la capacità espressiva del modello. Mentre l'approccio tradizionale predilige lo spazio euclideo per la sua trattabilità computazionale è possibile anche lavorare sul piano iperbolico esattamente come con le GCN; l'essenza del KGE sta nel definire come i vettori costruiti su questo piano latente interagiscano per rappresentare al meglio la validità delle informazioni.

Della letteratura e dei modelli allo stato dell'arte emergono due modi concettualmente distinti di interpretare l'embedding delle relazioni:

1. Relazioni come Vettori (Modelli Traslazionali): in questa visione, la relazione è intesa come una traslazione geometrica nello spazio latente. Se una tripla (h,r,t) è vera, il vettore della relazione r agisce come un ponte che "muove" l'embedding della testa h verso la posizione occupata dall'embedding della coda t.
   In maniera formale questa proprietà è definita come $h + r \approx t$.

   Modelli noti in letteratura che rispecchiano questo approccio sono TransE e MurE.

2. Relazioni come operatori appresi: una prospettiva più sofisticata che vede le relazioni non solo come semplici punti o vettori di spostamento, ma come operatori funzionali che trasformano gli embedding delle entità. In questo caso, r agisce come una funzione non lineare applicata a h per proiettarlo in una regione dello spazio dove la verosimiglianza di incontrare t sia massima. Questa interpretazione permette una flessibilità significativamente maggiore, consentendo al modello di catturare interazioni non lineari e asimmetrie semantiche.

   Modelli noti in letteratura che rispecchiano questo approccio sono RotatE, RefE e AttE.

L'apprendimento degli embedding non avviene in modo isolato, ma segue un obiettivo finale, come ad esempio classificazione dei nodi o link prediction. Il sistema non apprende solo "dove" posizionare i nodi, ma apprende a discriminare la verità.

Dato un set di triple di addestramento E_{train}, il modello ottimizza i parametri affinché una tripla corretta (h, r, t) restituisca la massima verosimiglianza (likelihood) rispetto a qualsiasi altra combinazione di nodi. Il fulcro computazionale è la funzione di punteggio:

$$s: V \times R \times V \rightarrow \mathbb{R} \tag{2}$$

Questa funzione agisce come un misuratore di probabilità: un punteggio elevato indica una forte probabilità che la relazione r esista tra h e t. Durante l'addestramento, la funzione s viene calibrata affinché lo scarto tra le triple reali e quelle generate artificialmente (tramite negative sampling) sia massimizzato.

3 SPERIMENTAZIONI

3.1 Schema metodologico

In questa sezione si analizza il contributo sperimentale della ricerca, con un focus specifico sui modelli HGCN e sulle modalità di integrazione con rappresentazioni basate su knowledge graph embedding (KGE).

Le sperimentazioni sono state organizzate secondo la seguente procedura sperimentale:

- Primi esempi di graph embedding e introduzione al modello Poincaré;
- Studio del modello GCN/HGCN su dataset medici noti e creati artificialmente, con l'obiettivo di analizzare l'impatto degli iperparametri;
- Applicazione pratica di modelli GCN/HGCN con KG contenente informazioni su malattie mendeliane, con l'obiettivo di supportarne la diagnosi;
- Confronto tra modelli con apprendimento su grafo omogeneo (GCN/HGCN) e knowledge graph embedding su grafo eterogeneo (AttE/AttH), analizzando costi e performance del training;
- Studio sull'impatto delle informazioni contenute nel KG delle malattie mendeliane, per analizzare quali informazioni portano migliorie nelle performance.

3.2 Librerie e dataset

Durante la ricerca sono state utilizzate le seguenti librerie:

- **NetworkX** [8]: rappresenta lo standard per la teoria dei grafi in ambiente Python. È stata utilizzata per la fase di pre-processing, permettendo la creazione, la manipolazione e l'analisi strutturale dei grafi prima della fase di embedding.
- **Node2Vec** [4] e **Gensim** [9] (in particolare il metodo PoincareModel): librerie impiegate per l'implementazione dei primi modelli di graph embedding, permettendo di apprendere rappresentazioni vettoriali in grado di catturare le proprietà strutturali dei nodi.
- **SciKit-Learn** [10]: libreria essenziale per il machine learning in Python, è stata sfruttata principalmente per le fasi di post-processing e valutazione. In particolare in particolare i metodi t-SNE e PCA per la visualizzazione bidimensionale di embedding complessi, preservando le relazioni di prossimità locale tra i nodi.
- la libreria **hgcn** [7]: questa libreria fornisce l'implementazione di riferimento per le GCN e HGCN, consentendo la creazione di modelli con un'ampia scelta di iperparametri.
- **KGEmb** [11]: framework specializzato per il Knowledge Graph Embedding. La libreria è stata cruciale per confrontare modelli allo stato dell'arte come AttE e AttH, permettendo di analizzare come i meccanismi di attenzione possano beneficiare della curvatura negativa per modellare relazioni complesse.

Per le prime sperimentazioni pratiche sono stati utilizzati 3 principali dataset:

- **PubMed** [12]: un dataset di reti di citazioni ampiamente utilizzato come benchmark per modelli di graph embedding. Consiste in un grafo dove i nodi rappresentano pubblicazioni scientifiche estratte dal database PubMed (relative alla ricerca sul diabete) e gli archi indicano le citazioni reciproche tra gli articoli.

- **CoRA** [13]: uno dei dataset standard per lo studio delle Graph Neural Networks. Anche questo tratta di una rete di citazioni in cui i nodi sono articoli di ricerca classificati in sette categorie tematiche (ad esempio, ”Neural Networks”, ”Reinforcement Learning”); ideale per valutare la capacità dei modelli di catturare sia la struttura del grafo che le feature semantiche dei nodi.

- Albero bilanciato creato con NetworkX fissando i parametri r=3 e h=6: struttura sintetica creata artificialmente, per testare i modelli in un ambiente a gerarchia perfetta. Poiché gli alberi presentano una crescita esponenziale del numero di nodi rispetto alla distanza dalla radice, questo grafo costituisce il caso di studio ideale per dimostrare la superiorità degli embedding iperbolici rispetto a quelli euclidei nel rappresentare strutture gerarchiche con una distorsione minima.

Dataset	**Nodi**	**Archi**	**Degree medio**	**Degree massimo**	**Degree minimo**
CoRA	2 708	5 278	3.89	168	1
PubMed	19 717	44 327	4.49	171	1
Albero Bilanciato	1 093	1 092	1.99	4	1

Table 1: Caratteristiche strutturali dei dataset utilizzati per la valutazione dei modelli.

3.2.1 Knowledge Graph per malattie mendeliane

La nostra ricerca si è concentrata principalmente sullo studio di un knowledge graph contenente informazioni mediche sulla diagnosi delle malattie mendeliane, prodotto da Anacleto-Lab [14](UniMi). Il grafo è stato costruito prendendo ispirazione dalle entità rappresentate nel dataset pubblico PrimeKG [15], un knowledge graph orientato alla medicina di precisione.

Utilizzando quindi PheKnowLator (Phenotype Knowledge Translator) [16], un framework specializzato per la costruizione ottimizzata di KG biomedici complessi e di larga scala, sono state selezionate delle ontologie categorizzate per tipo di nodo. Le tipologie di nodi che formano il grafo finale sono i seguenti:

- nodo **Protein**:

  rappresenta Protein Ontology (PR), Cell Ontology (CL), Human Phenotype Ontology (HP) e Molecular Function (MF);

- nodo **GO**:

  rappresenta Gene Ontology Chemical Entities (GOCHE) e Gene Ontology (GO);

- nodo **Disease**:

  rappresenta Disease Ontology (DOID) e Mondo Disease Ontology (MONDO);

- nodi **Gene** e **Genomic features**:

  rappresentano Ontology of Genes and Genomes (OGG) e Sequence Ontology (SO);

- nodo **Phenotype**:

  rappresenta Human Phenotype Ontology (HP), Phenotype And Trait Ontology (PATO) e uPheno Ontology (UPHENO);

- nodo **Person**:

  rappresenta i pazienti, composto da due sottonodi che rappresentano rispettivamente il sesso maschile e quello femminile;

Successivamente, il grafo è stato ampliato includendo nodi aggiuntivi per rappresentare ulteriori patologie. In particolare, i dati dei pazienti sono stati raccolti secondo il Phenopacket Schema [17], consentendo l'introduzione di nodi di tipo *Article* al fine di stabilire connessioni semantiche tra pazienti affetti da patologie comuni.

Il risultante grafo finale è composto da 2 723 540 archi e 243 053 nodi, una rappresentazione schematica è presente nella Figura 10.

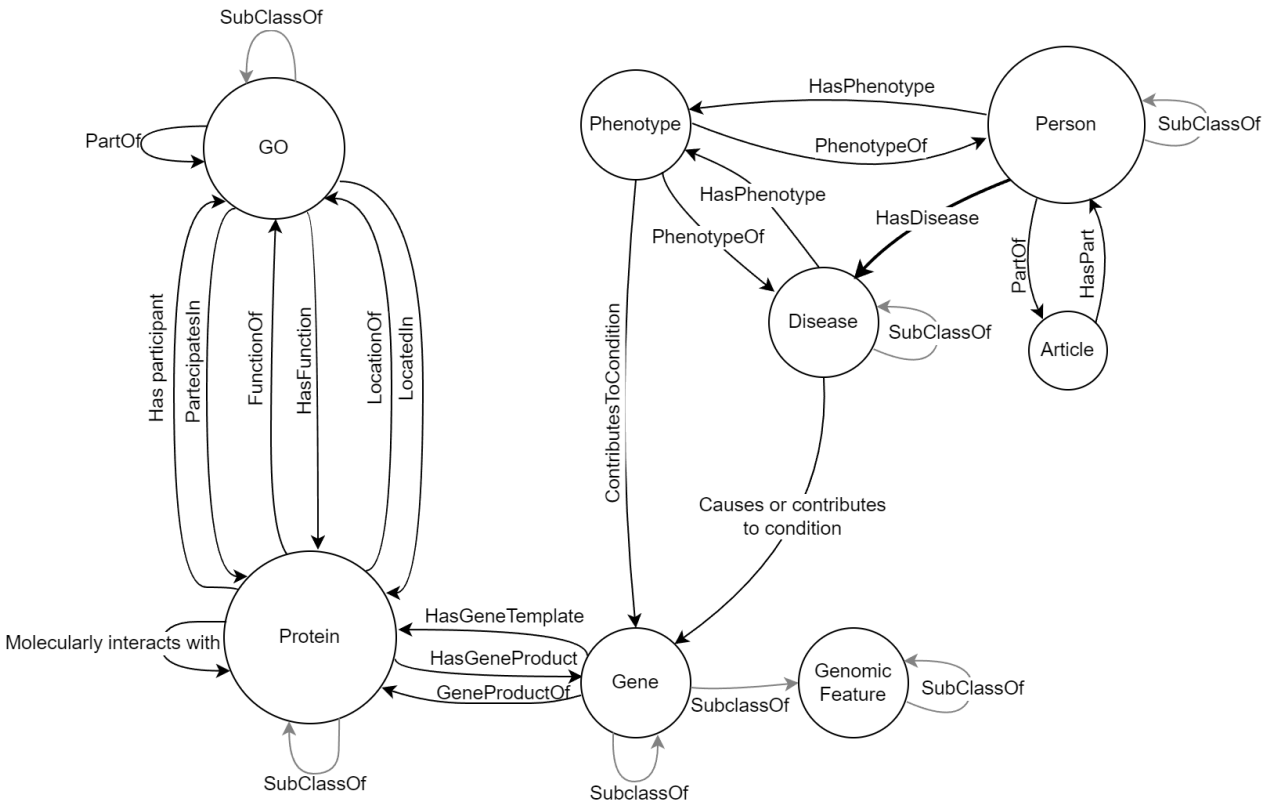


Figure 10: Rappresentazione schematica generale del grafo finale, comprende tutte le tipologie di nodi coinvolti, ma solamente gli archi più significativi

La composizione del grafo evidenzia forti relazioni gerarchiche, dovute principalmente alla presenza di sottocategorie di nodi: l'arco di tipo *SubClassOf* definisce infatti una gerarchia tra nodi omogenei, che si articola dal livello più generico (radice) fino a concetti sempre più specifici (foglie), formando così una struttura ad albero in corrispondenza delle principali categorie.

Questa forte gerarchia rafforza la scelta di adottare modelli basati su spazio iperbolico (come HGCN), noti per la loro elevata capacità di rappresentare strutture gerarchiche in modo più efficiente rispetto ai tradizionali approcci basati su spazio euclideo.

Tipo di arco	**Subject**	**Object**	**#**
Molecularly interacts with	Protein	Protein	618 856
Phenotype of	Phenotype	Disease	453 260
Has phenotype	Disease	Phenotype	453 260
Subclassof	Protein	Protein	164 224
Has participant	Protein	GO	125 233
Participates in	GO	Protein	125 230
Located in	GO	Protein	81 468
Subclassof	GO	GO	81 608
Location of	Protein	GO	81 466
Has function	GO	Protein	68 742
Function of	Protein	GO	68 742
Subclassof	Disease	Disease	45 251
Subclassof	Gene	Genomic feature	28 130
Subclassof	Phenotype	Phenotype	25 852
Causes/contributes to condition	Phenotype	Gene	24 519
Has gene template	Gene	Protein	20 089
Gene product of	Gene	Protein	19 607
Has gene product	Protein	Gene	19 607
Causes/contributes to condition	Disease	Gene	14 418
Has disease	**Person**	**Disease**	**7 799**
Part of	GO	GO	6 772

Table 2: Numero di archi più significativi raggruppati per tipologia e per nodi coinvolti

La tabella 2 mostra il tipo di archi che compongono il grafo, elencati in ordine per cardinalità. Questo evidenzia una prevalenza delle interazioni con i nodi *Protein*, seguita dalle relazioni tra *Disease* e *Phenotype*.

Questa distribuzione riflette la centralità delle proteine nei processi biologici e patologici: esse costituiscono i principali componenti funzionali della cellula e sono direttamente coinvolte nelle attività biologiche fondamentali, incluse le interazioni molecolari alla base delle malattie. Di conseguenza, è atteso osservare un'elevata densità di relazioni che coinvolgono proteine, poiché alterazioni a livello proteico (in termini di struttura, funzione o regolazione) costituiscono uno dei principali determinanti delle condizioni patologiche.

Allo stesso modo, la forte presenza di connessioni tra *Disease* e *Phenotype* è coerente con la pratica clinica, in cui le malattie vengono definite e classificate proprio sulla base delle manifestazioni fenotipiche osservabili nel paziente. I fenotipi rappresentano quindi il livello descrittivo attraverso cui le alterazioni molecolari si traducono in segni e sintomi clinici, rappresentando una base per le operazioni di diagnosi.

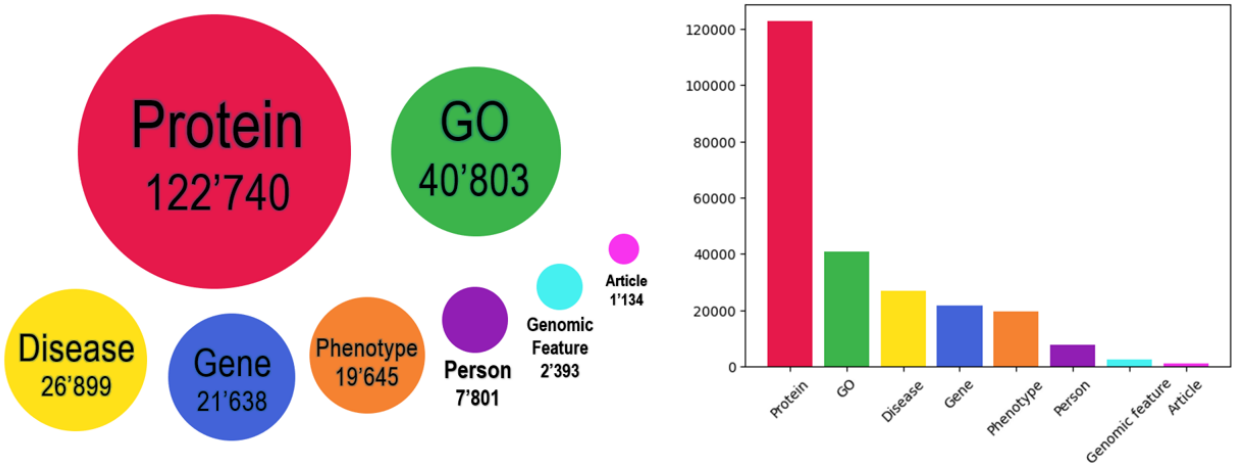


Figure 11: Rappresentazione grafica delle proporzioni dei nodi raggruppati per tipologia

Dalla Figura 11 si può notare che il knowledge graph fornisce un'integrazione sia a livello molecolare sia a livello clinico: da un lato le proteine contribuiscono a descrivere i meccanismi biologici, dall'altro le malattie e i fenotipi permettono di collegare tali meccanismi alle manifestazioni osservabili nei pazienti. Nel complesso, emerge una struttura bilanciata tra componenti biologiche e cliniche, utile per analizzare le relazioni tra cause molecolari e effetti fenotipici.

3.3 Metriche

L'analisi delle performance dei modelli è stata fatta utilizzando metriche specifiche relative al tipo di task su cui si basava il training.

Nelle prime sperimentazioni con i modelli omogenei GCN e HGCN sono state valutate le loro performance sul task di link prediction. Per questo tipo di task sono state utilizzate le metriche di ROC-AUC e accuracy.

La Area Under the Receiver Operating Characteristic Curve (ROC-AUC) è una metrica che valuta la capacità del modello di distinguere tra due classi (presenza o assenza di un arco). Rappresenta la probabilità che il modello assegni un punteggio di confidenza (score) più elevato a un arco esistente (positivo) scelto casualmente rispetto a un arco non esistente (negativo) scelto casualmente, formalmente:

$$AUC = \frac{1}{|E_{pos}| \cdot |E_{neg}|} \sum_{e_{pos} \in E_{pos}} \sum_{e_{neg} \in E_{neg}} \mathbb{I}(s_{e_{pos}} > s_{e_{neg}})$$

con s la funzione di score definita nel modello GCN (Equazione 1).

La metrica di accuracy è stata valutata sui valori di score prodotti su vari test set che sono stati costruiti mantenendo un bilanciamento tra classe positiva (arco presente) e classe negativa (arco non presente), in modo da non distorcere l'efficacia dell'informazione data dall'accuracy. Formalmente è definita come:

$$\text{Accuracy} = \frac{\sum_{(u,v) \in E_{test}} \mathbb{I}(s_{uv} = y_{uv})}{|E_{test}|}$$

dove $y_{uv} \in \{0, 1\}$ è la ground truth (1 se l'arco esiste, 0 altrimenti), mentre s_{uv} è lo score della link prediction prodotta dal modello sulla coppia di nodi u e v.

Nella seconda parte della sperimentazione lo studio si è concentrato sull'analizzare le performance dei modelli sia di tipo omogeneo (GCN/HGCN) sia di tipo eterogeneo (AttE/AttH), valutando non più solo l'efficacia delle predizioni sul task di link prediction, ma espandendo l'analisi sulla qualità delle predizioni prodotte.

Sono state quindi valutate le prestazioni dei modelli utilizzando alcune metriche strutturali comunemente impiegate nei task di link prediction su knowledge graph, come Mean Rank (MR), Mean Reciprocal Rank (MRR) e Hits@1 (H@1).

$$
\begin{aligned}
\text{MR} &= \frac{1}{|\mathscr{Q}|} \sum_{i=1}^{|\mathscr{Q}|} \text{rank}_i \\
\text{MRR} &= \frac{1}{|\mathscr{Q}|} \sum_{i=1}^{|\mathscr{Q}|} \frac{1}{\text{rank}_i} \\
\text{Hits@k} &= \frac{1}{|\mathscr{Q}|} \sum_{i=1}^{|\mathscr{Q}|} \mathbf{1}\left(\text{rank}_i = k\right)
\end{aligned}
\tag{3}
$$

con $\mathscr{Q}$ l'insieme delle predizioni effettuate sul test set.

Queste metriche consentono di valutare in modo preciso la qualità delle predizioni dei modelli. In particolare, il Mean Rank (MR) indica la posizione (rank) media in cui compare la malattia corretta tra le predizioni generate per ciascun paziente. La metrica Hits@k (H@k), invece, misura la percentuale di casi in cui il modello colloca la malattia corretta entro il K-esimo (compreso) posto.

3.4 Studio preliminare delle HGCN su dataset di letteratura medica

La prima sperimentazione è stata fatta utilizzando le HGCN su 3 principali dataset: i primi 2 forniti all'interno dalla libreria, ovvero Cora e PubMed, dataset ampiamente utilizzati come benchmark per valutare modelli di classificazione, e il terzo dataset è un grafo creato artificialmente che rappresenta un albero bilanciato.

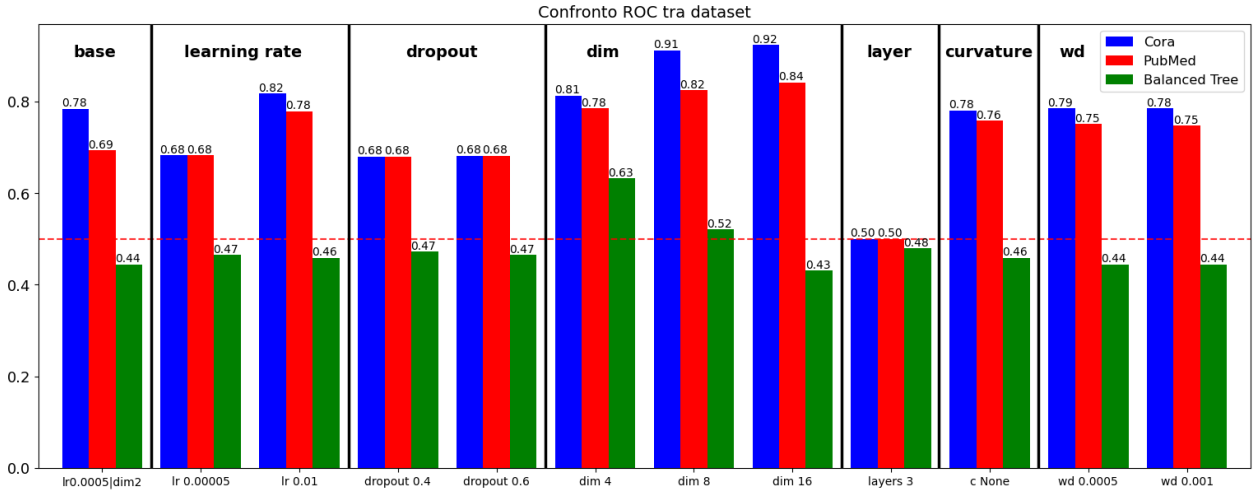


Figure 12: Andamento della metrica ROC-AUC al variare degli iperparametri sui 3 dataset.

Per ciascun dataset è stata eseguita una procedura di grid search, ripetendo l'addestramento del modello con diverse combinazioni di iperparametri. Se non coinvolti i parametri di dimensionalità e il learning rate sono stati mantenuti con valore rispettivo 2 e 0.0005. I risultati ottenuti sono stati poi rappresentati mediante un grafico a barre (Figura 12), al fine di evidenziare le differenze di prestazioni tra le varie configurazioni.

Il risultato che emerge con maggiore evidenza è che alcuni iperparametri, in particolare il learning rate e la dimensionalità dei vettori di embedding, influenzano in modo significativo le prestazioni dei modelli. Questo comportamento si riscontra in maniera consistente su tutti e tre i dataset considerati, suggerendo che la scelta accurata di questi parametri riveste un ruolo cruciale nelle performance.

3.5 Confronto pratico HGCN vs GCN su dataset di letteratura medica

Nella seconda sperimentazione l'obiettivo è stato quello di confrontare le prestazioni del modello HGCN rispetto a GCN, evidenziandone le differenze di comportamento. Per fare ciò sono state valutate diverse configurazioni di apprendimento, variando i parametri più influenti riscontrati dalla prima sperimentazione, ovvero il learning rate e la dimensione dell'embedding, così da analizzare ulteriormente l'impatto di tali iperparametri sulle performance dei due modelli. Anche in questo caso gli esperimenti sono stati fatti sui precedenti 3 dataset, utilizzando esclusivamente la struttura dei grafi.

Questa seconda sperimentazione, inoltre, non ha previsto l'uso di feature associate ai nodi. Per semplicità verrà considerata solamente la metrica di ROC-AUC nel confronto delle performance.

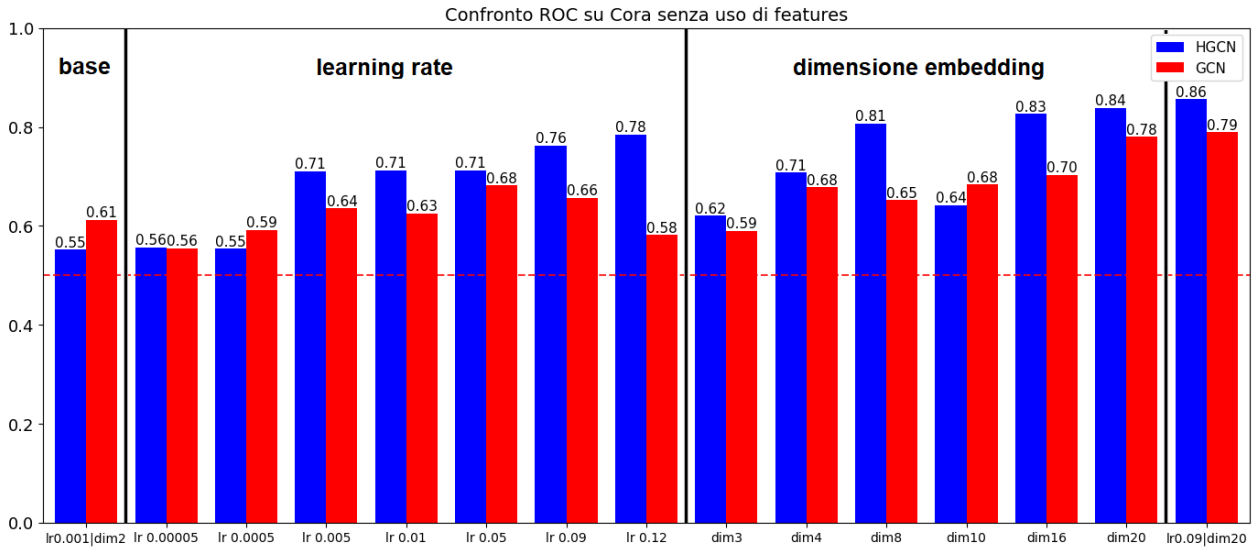


Figure 13: Confronto sul dataset Cora, HGCN guadagna mediamente il **6.70%** nelle performance rispetto a GCN

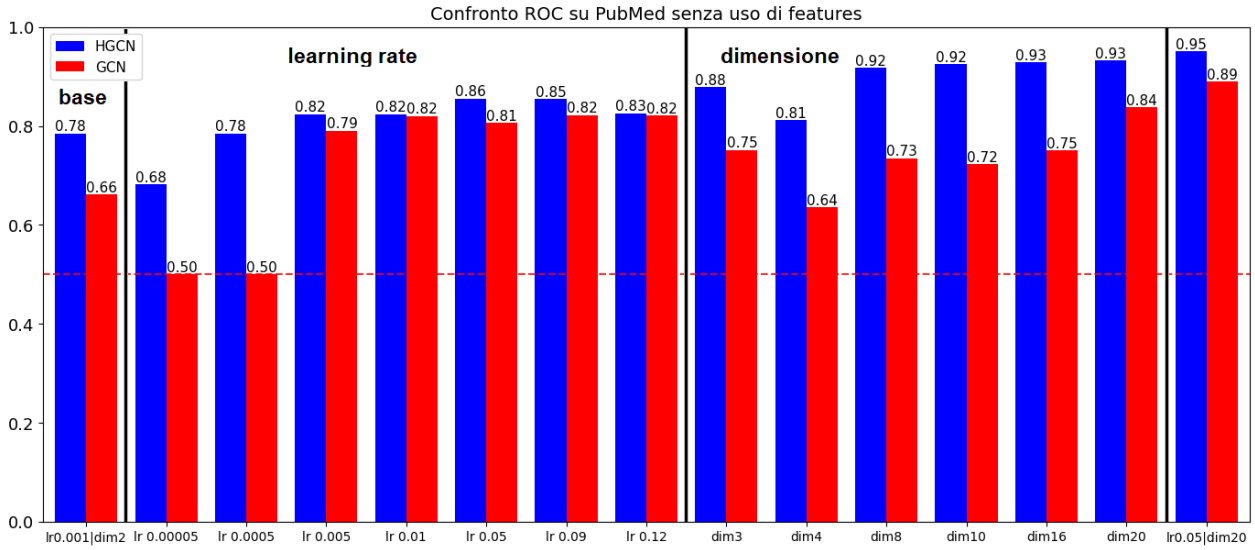


Figure 14: Confronto sul dataset PubMed, HGCN guadagna mediamente il **13.79%** nelle performance rispetto a GCN

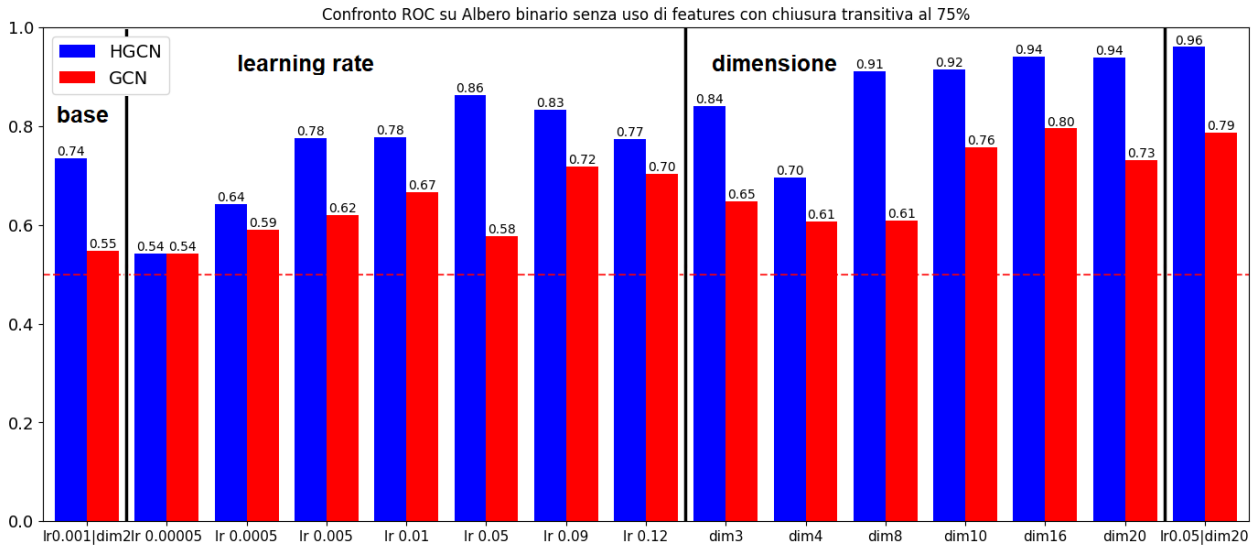


Figure 15: Confronto sul dataset di un albero bilanciato con modifica della chiusura transitiva (75%), HGCN guadagna in media il **17.77%** nelle performance ripetto a GCN

I confronti riportati nelle Figure [13, 14, 15] evidenziano prestazioni complessivamente superiori del modello HGCN rispetto a GCN, confermando le considerazioni discusse in precedenza. Nella Figura 15, che rappresenta le performance dei modelli su un albero 3-ario bilanciato di altezza 6 (1092 achi e 1093 nodi), è stata applicata la chiusura transitiva al 75% per garantire un corretto training dei modelli.

Questa modifica è legata a una problematica che si verifica nella fase di addestramento. La struttura del grafo infatti rappresenta un albero ideale, privo delle irregolarità tipiche dei grafi reali, quali cicli locali e variabilità nel grado dei nodi. Di conseguenza, durante la fase di training e più precisamente nella selezione degli archi destinati al test set, la rimozione anche di un singolo arco può causare la frammentazione del grafo in più componenti connesse non comunicanti tra loro. Questo fenomeno compromette la capacità del modello di apprendere e inferire correttamente la struttura globale del grafo.

Per risolvere questa problematica è stato necessario introdurre una modifica strutturale del grafo, aggiungendo archi tramite la chiusura transitiva. Nella teoria dei grafi, la chiusura transitiva consiste nel collegare direttamente ciascun nodo con tutti i nodi appartenenti alle generazioni successive dei suoi sottofigli, aumentando così la connettività locale e globale del grafo.

Al fine di individuare la strategia più efficace, è stata condotta un'ulteriore sperimentazione volta ad analizzare l'impatto del numero di archi aggiunti mediante chiusura transitiva. In particolare, le performance dei modelli sono state valutate secondo le stesse modalità descritte in precedenza, confrontando i risultati al variare della percentuale di archi aggiunti nella chiusura transitiva rispetto alla struttura originale del grafo.

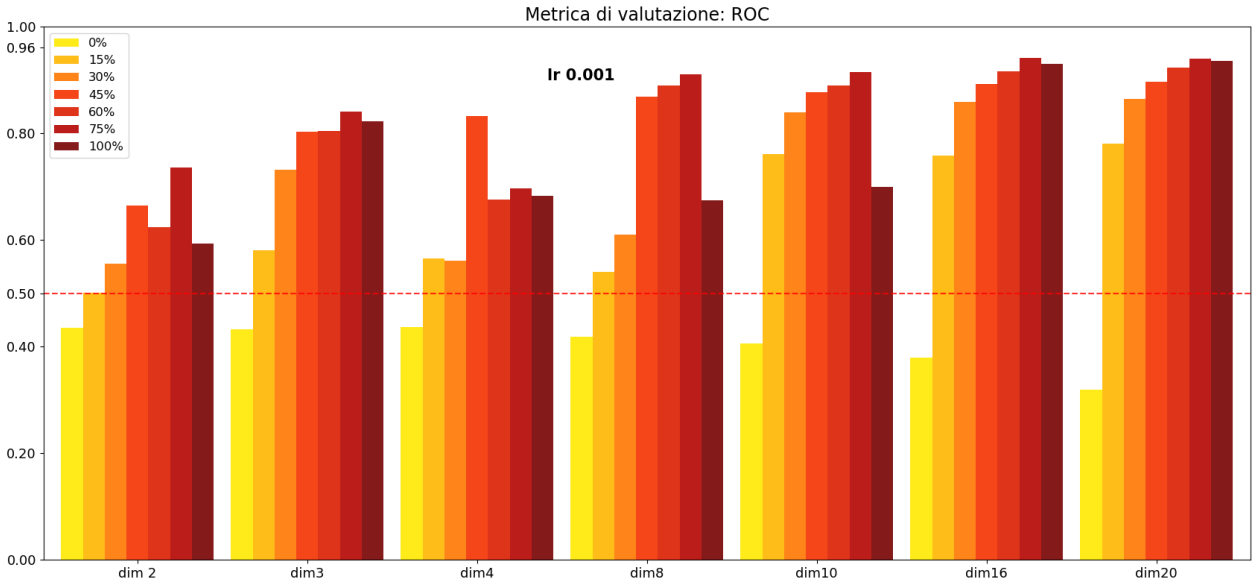
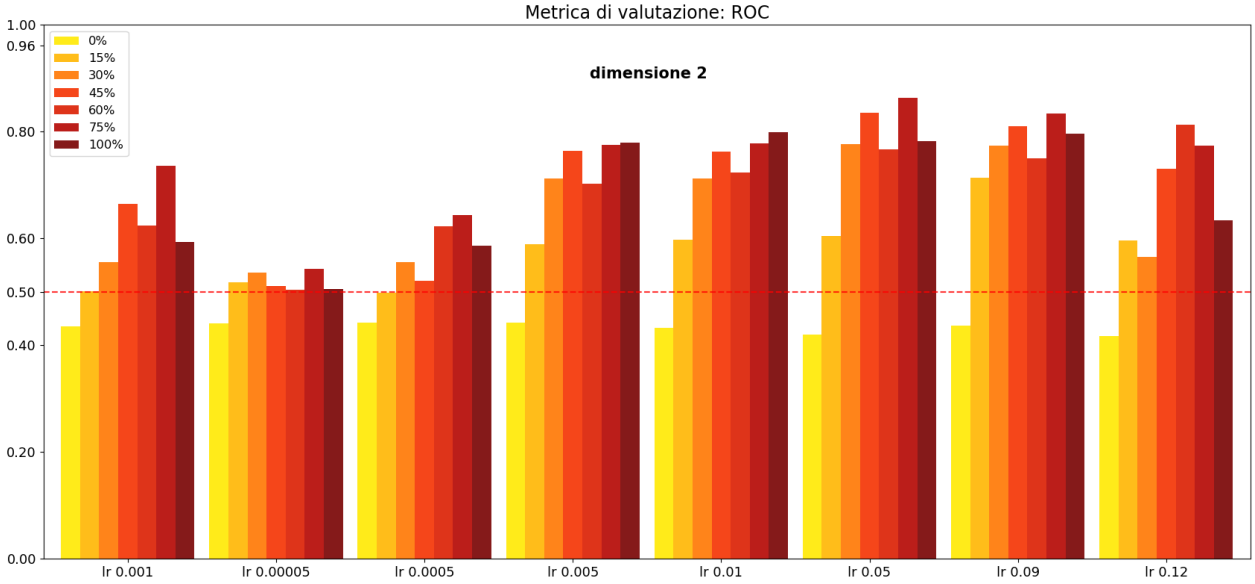


Figure 16: Confronto delle performance ottenute su HGCN con vari livelli di chiusura transitiva



Grazie a questa analisi è stato possibile studiare il compromesso tra arricchimento strutturale del grafo e capacità di generalizzazione del modello. Anche qua per semplicità è stata considerata solamente la metrica ROC-AUC e solamente i risultati ottenuti sul modello HGCN.

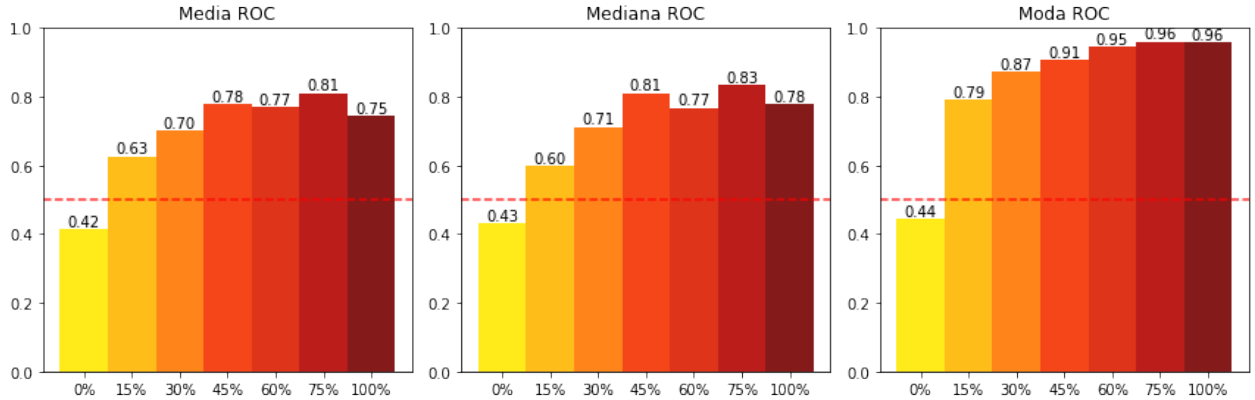


Figure 17: Confronto metriche ottenute a varie percentuali di chiusura transitiva

I risultati (Figure 16, 17) mostrano le migliori performance quando vengono aggiunti il 75% degli archi della chiusura transitiva tra la radice e tutti i sottofigli, raggiugendo discrete capacità di apprendimento; il confronto diretto tra i 2 modelli, anche su questo tipo di dataset, ha mostrato un netto guadagno nelle metriche del modello HGCN rispetto a GCN.

Questi risultati confermano quindi una capacità migliore di rappresentazione da parte del modello HGCN rispetto a GCN, in particolare viene nuovamente confermato un significativo guadagno su grafi di natura gerarchica.

3.6 Caso studio: knowledge graph per malattie mendeliane

3.6.1 Motivazioni della sperimentazione

La ricerca si è concentrata sullo studio della diagnosi delle malattie mendeliane [18], ovvero patologie ereditarie causate dalla mutazione di un singolo gene, con l'obiettivo di supportare la predizione di queste malattie.

In ambito clinico, la diagnosi di queste patologie risulta tuttora particolarmente complessa per diversi fattori. Nonostante la disponibilità di tecniche avanzate di sequenziamento dell'esoma e del genoma, oltre il 50% dei pazienti non riceve una diagnosi molecolare ben definita [1]. Ciò è attribuibile a una comprensione ancora incompleta della patogenicità delle varianti genetiche, alla mancanza di associazioni consolidate tra genotipo e fenotipo e alle differenze nei criteri di interpretazione e refertazione adottati dai diversi laboratori clinici. Inoltre, mutazioni nello stesso gene possono manifestarsi con fenotipi differenti, oppure uno stesso paziente può presentare simultaneamente più condizioni genetiche, rendendo il processo di profiling più complesso.

Recenti studi dimostrano che la creazione di dataset basati su reti eterogenee (i.e. gene-fenotipo-malattia) influenzano in maniera molto rilevante le performance di modelli allenati su di esse [19]. Questo approccio richiede tuttavia architetture in grado di apprendere non soltanto dalle singole feature delle entità, ma anche dalle relazioni strutturali che le collegano. In questo contesto, le Graph Convolutional Neural Network (GCN) rappresentano una delle soluzioni più appropriate, poiché consentono di modellare e sfruttare efficacemente le dipendenze topologiche all'interno del grafo.

I modelli di apprendimento automatico applicati a grafi eterogenei rappresentano quindi un approccio efficace per integrare dati genomici e fenotipici attraverso relazioni strutturate. Questo approccio consente di sfruttare simultaneamente le caratteristiche delle singole entità e le dipendenze topologiche che le connettono, offrendo il potenziale di migliorare significativamente la predizione delle associazioni tra varianti genetiche e malattie rispetto all'analisi isolata delle varianti o ai metodi di machine learning tradizionali.

3.6.2 Applicazione dei modelli GCN/HGCN sul KG delle malattie mendeliane

La prima sperimentazione fatta su questo knowledge graph è stata allenare i modelli di HGCN e GCN con l'obiettivo di apprendere la relazione *Has disease* tra *Person* e *Disease*, per confrontare le performance ottenute dai due modelli. Il training dei modelli è stato eseguito utilizzando le risorse di calcolo ad alte prestazioni del cluster INDACO dell'Università degli Studi di Milano; l'ambiente è stato configurato mettendo a disposizione una RAM di 300GB, una GPU A100 da 80GB e un processore AMD EPYC 9354 32-Core.

Nonostante l'hardware molto performante, la libreria hgcn [7] ha mostrato inefficienza nell'uso della memoria, per questo motivo non è stato possibile utilizzare interamente il grafo per il training dei modelli. Per risolvere questa problematica è stata utilizzata una versione ridotta del grafo originale, il cui schema è rappresentato dalla Figura 18, composto quindi solamente da 28 811 nodi e 178 320 archi.

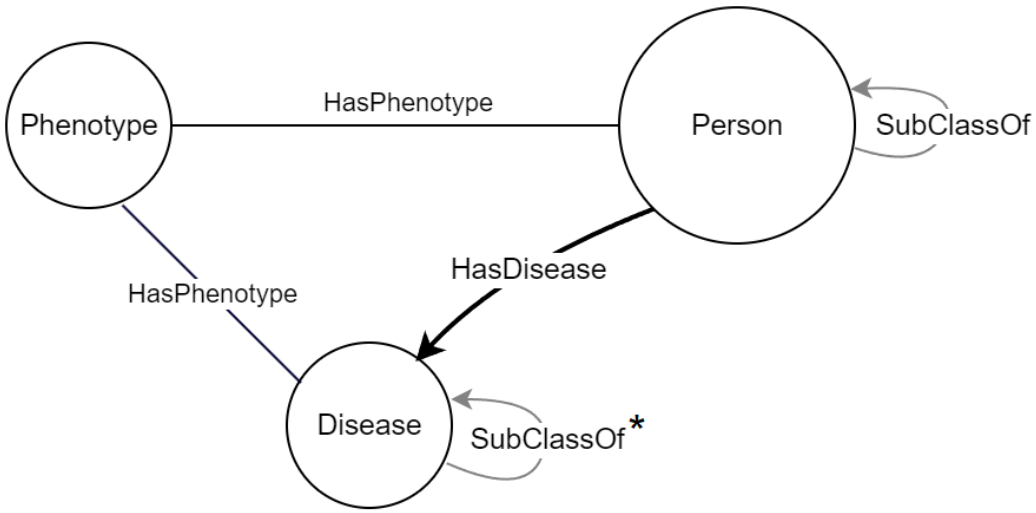


Figure 18: Grafico finale ridotto

Il grafo ridotto è costituito dal nucleo di nodi di tipo *Person*, *Disease* e *Phenotype*. Il numero di nodi relativi alle malattie è stato ridotto: tra le 26 899 malattie sono state mantenute esclusivamente quelle associate ad almeno un paziente (489) e le rispettive malattie padre raccolte fino alla radice (879), derivate dalla relazione gerarchica *SubClassOf*, ottenendo un totale finale di 1 368 malattie. Anche gli archi sono stati semplificati: è stata eliminata la relazione *PhenotypeOf*, mantenendo unicamente *HasPhenotype*. Questa scelta non altera la semantica del grafo, poiché la libreria utilizzata consente di rappresentare le relazioni come bidirezionali.

Il dataset utilizzato per l'addestramento dei modelli è stato costruito a partire dalla versione ridotta del grafo. Su questa base è stato definito uno split in train, validation e test (tabella 3). Il test set è stato ottenuto selezionando le 30 malattie con il maggior numero di associazioni; da queste, sono stati estratti casualmente gli archi coinvolti, assegnando a ciascuna malattia una probabilità proporzionale al proprio tasso di associazione.

Split	**Numero di archi**	**%**
Train	168 624	94.5
Validation	8 136	4.5
Test	1 560	1.0
Totale	178 320	100

Table 3: Suddivisione del dataset in train, validation e test set

Successivamente sono stati generati i negative samples utilizzando le malattie padre, che non presentano associazioni dirette con i pazienti. Questo approccio ha permesso di ottenere un test set bilanciato tra esempi positivi e negativi.

Il test set risulta relativamente ridotto perché rappresenta il 10% degli archi totali di tipo *HasDisease* (7 799 in totale). A questi sono stati aggiunti gli ulteriori 780 archi negativi, sempre considerando gli archi *HasDisease*, per il bilanciamento del test set.

La porzione train + validation comprende infine tutti gli archi del grafo che non sono stati coinvolti nella costruzione del test set.

La suddivisione degli split è stata effettuata con l'obiettivo di preservare il più possibile le informazioni utili all'apprendimento. In particolare, il validation set è stato utilizzato esclusivamente per monitorare l'andamento della loss durante l'addestramento e determinare il numero massimo di epochs necessarie per la convergenza.

3.6.3 Training e risultati

La fase di training è stata organizzata attraverso un'analisi sistematica del comportamento dei due modelli al variare di specifici iperparametri. In particolare, sono state considerate tutte le combinazioni tra due insiemi di valori: un set di learning rate pari a [0.005, 0.1, 0.9] e un set di dimensioni dell'embedding pari a [2, 4, 10, 26]. Questa configurazione ha portato alla definizione di 12 esperimenti distinti, corrispondenti a tutte le possibili combinazioni tra i valori dei due iperparametri.

Per ciascuna configurazione, il modello è stato addestrato per un totale di 1000 epochs. La scelta di questo numero di epochs è stata determinata empiricamente osservando l'andamento della funzione di loss e delle metriche di validazione sul validation set, al fine di individuare un punto in cui l'apprendimento mostrasse una chiara convergenza. In questo modo è stato possibile garantire che il training fosse sufficientemente lungo da consentire al modello di stabilizzare le proprie prestazioni senza introdurre un eccessivo rischio di overfitting.

Al fine di valutare la stabilità dei modelli rispetto alla variabilità introdotta dai processi di addestramento, ciascun esperimento è stato ripetuto utilizzando tre seed differenti. In queste ripetizioni il test set è stato randomizzato mantenendo la scelta delle 30 malattie più associate, mentre i dataset di training e validation sono stati rimescolati tra loro in base al seed utilizzato. Questo approccio ha permesso di verificare la robustezza dei risultati ottenuti e di ridurre il rischio che le prestazioni osservate fossero influenzate da una specifica suddivisione dei dati.

L'analisi dei risultati ottenuti nelle diverse configurazioni ha quindi permesso di valutare l'impatto dei due iperparametri considerati sulle prestazioni complessive dei modelli.

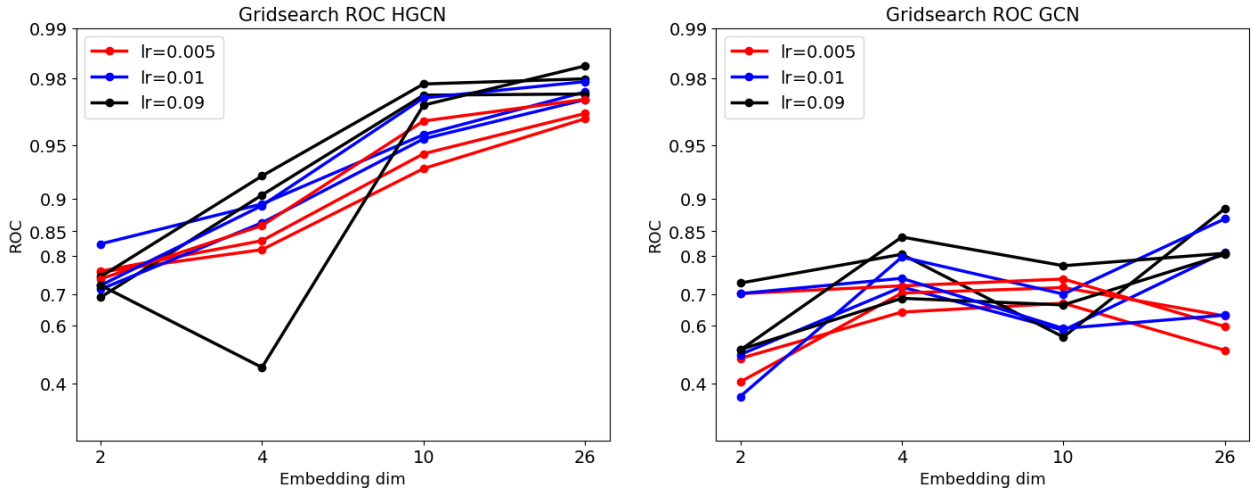


Figure 19: Curva ROC dei modelli sul gridsearch effettuato con 3 seed differenti

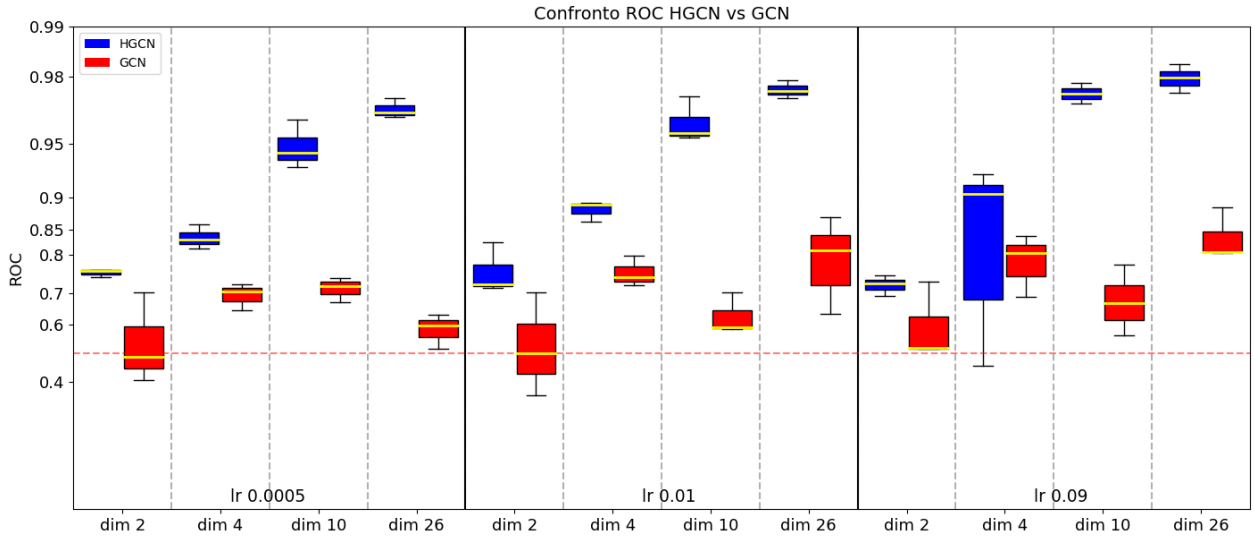


Figure 20: Confronto diretto tra modelli su boxplot con 3 seed raggruppati per iperparametri

Le figure 19 e 20 mostrano un'analisi delle performance ottenute facendo riferimento solamente alla curva ROC sul task di link prediction.

Si può osservare come il modello HGCN mostri un trend di miglioramento all'aumentare della dimensione dell'embedding, con poca influenza del learning rate. Il modello GCN invece sembra non mostrare un chiaro trend, i risultati rimangono poco stabili anche al variare degli iperparametri, suggerendo difficoltà nell'apprendimento.

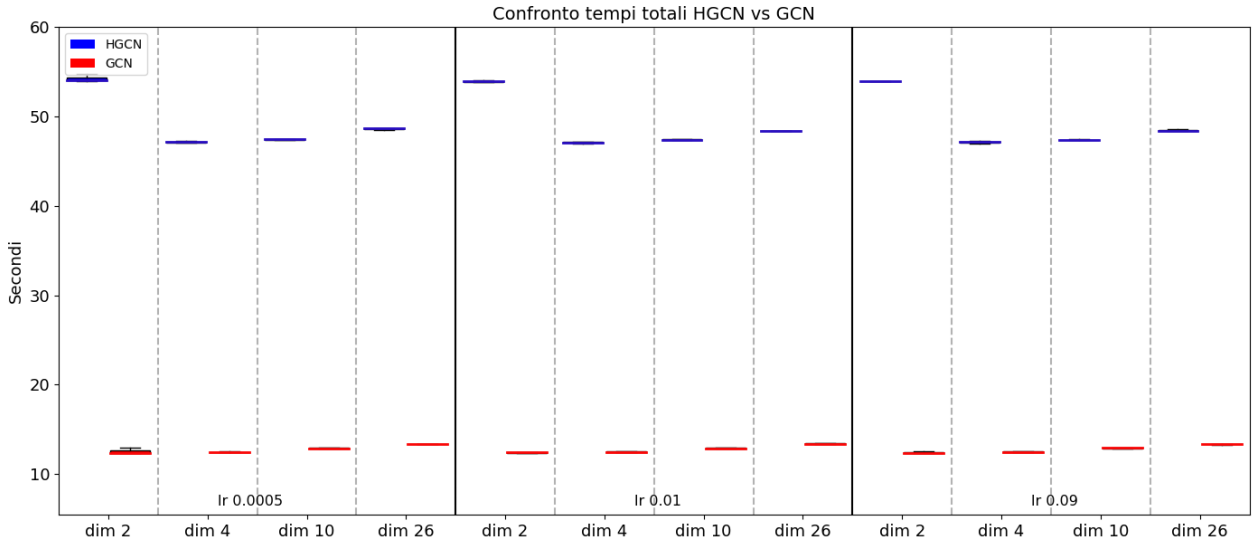


Figure 21: Confronto tra HGCN e GCN tramite gridsearch su tempi di esecuzione totale con 3 seed utilizzati

La Figura 21 mostra il tempo totale di esecuzione dei modelli, considerando solamente il tempo di calcolo della loss e delle metriche sul train set, escludendo quello necessario per il calcolo delle metriche sul validation set.

Si osserva come entrambi i modelli mantengono un tempo di esecuzione molto stabile; HGCN però risulta in media quasi 4 volte più lento rispetto a GCN (da circa 13 sec a 50 sec). Tuttavia, questo incremento computazionale è compensato da un miglioramento medio delle metriche con un guadagno di circa il 23% nella metrica ROC, 9% rispetto alla recall e 40% rispetto alla precision.

Questi ulteriori risultati dimostrano ancora una volta che la scelta dello spazio iperbolico incide in maniera molto positiva nelle performance, permettendo al modello HGCN di apprendere con ottimi risultati il task di link prediction sulle malattie mendeliane.

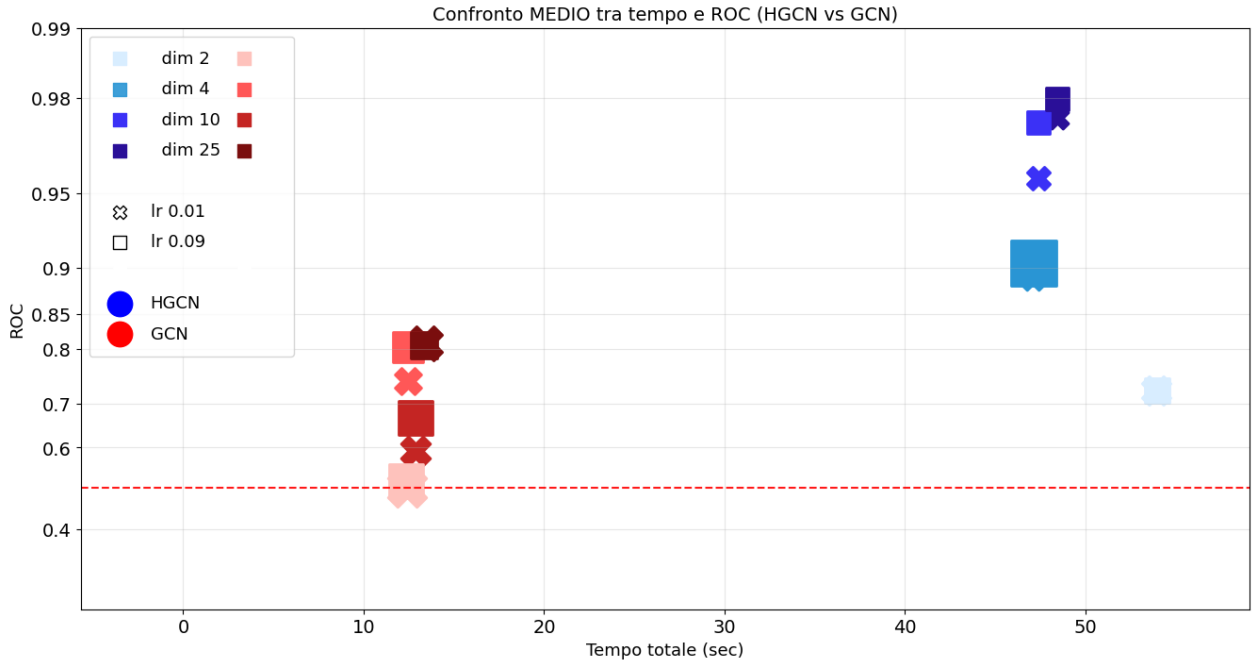


Figure 22: Confronto medio su 3 seed tra HGCN e GCN, la grandezza dei punti indica l'instabilità, quindi, una maggiore deviazione standard

Un'ulteriore analisi confronta le performance dei modelli con il tempo di esecuzione e la stabilità rispetto alla media su 3 differenti seed casuali, con l'obiettivo di valutare l'efficienza dei modelli in funzione della scelta degli iperparametri.

Dai risultati (Figura 22) emerge come configurazioni con dimensione dell'embedding superiore a 4 mostrino una maggiore stabilità tra le diverse esecuzioni. Tuttavia, a parità di configurazione, il modello GCN risulta significativamente meno efficiente rispetto a HGCN.

	dim 2		dim 4		dim 10		dim 26	
Metric	GCN	HGCN	GCN	HGCN	GCN	HGCN	GCN	HGCN
ROC-AUC	0.55 $\pm$ 0.02	**0.74** $\pm$ **0.02**	0.74 $\pm$ 0.02	**0.83** $\pm$ **0.10**	0.67 $\pm$ 0.02	**0.96** $\pm$ **0.00**	0.73 $\pm$ 0.03	**0.97** $\pm$ **0.00**
Precision	0.24 $\pm$ 0.01	**0.66** $\pm$ **0.01**	0.58 $\pm$ 0.01	**0.76** $\pm$ **0.04**	0.58 $\pm$ 0.02	**0.91** $\pm$ **0.01**	0.62 $\pm$ 0.03	**0.94** $\pm$ **0.00**
Recall	0.42 $\pm$ 0.01	**0.85** $\pm$ **0.00**	**0.91** $\pm$ **0.04**	0.79 $\pm$ 0.12	0.90 $\pm$ 0.01	**0.91** $\pm$ **0.01**	**0.94** $\pm$ **0.00**	0.92 $\pm$ 0.01
F1-score	0.30 $\pm$ 0.00	**0.74** $\pm$ **0.01**	0.70 $\pm$ 0.01	**0.77** $\pm$ **0.10**	0.70 $\pm$ 0.01	**0.91** $\pm$ **0.00**	0.75 $\pm$ 0.02	**0.93** $\pm$ **0.00**

Table 4: Confronto medio su 3 seed tra GCN e HGCN, con affiancata la relativa deviazione standard (in grassetto la metrica migliore)

3.6.4 Applicazione e analisi di metodi per grafi eterogenei

Fino ad ora, nel corso delle sperimentazioni sono stati utilizzati modelli di tipo Graph Convolutional, che tuttavia presentano una limitazione strutturale: lavorano principalmente su grafi omogenei, ovvero grafi in cui tutti i nodi e tutti gli archi appartengono a un unico tipo.

Questa caratteristica semplifica la struttura del grafo e le operazioni di convoluzione, ma riduce la capacità del modello di rappresentare relazioni più complesse presenti in molti scenari reali, soprattutto di tipo medico, nei quali sono presenti diverse tipologie di nodi e di relazioni tra di essi.

Per valutare l'impatto dell'utilizzo del knowledge graph dei pazienti mediante modelli eterogenei è stata utilizzata la libreria *KGEmb* [11]. Questa libreria implementa diversi modelli descritti nell'articolo *Low-Dimensional Hyperbolic Knowledge Graph Embeddings*, come RotE e RotH, AttH e AttE, oltre ad alcuni modelli considerati stato dell'arte come TransE.

Come modelli di riferimento in questa ricerca sono stati utilizzati AttE e AttH, due modelli che rappresentano le relazioni tra entità attraverso l'applicazione di diverse trasformazioni geometriche, in particolare rotazioni (rotation) e riflessioni (reflection). A ciascuna trasformazione viene assegnato un peso tramite un meccanismo di *attention*, che consente di combinarle in modo efficiente per modellare le relazioni.

La principale differenza tra AttE e AttH è analoga a quella tra GCN e HGCN: il primo modello opera in spazio euclideo, mentre il secondo utilizza uno spazio iperbolico. Questa distinzione consente di mantenere una comparazione diretta con i modelli precedentemente analizzati.

La configurazione di training adottata è la stessa utilizzata per i modelli omogenei, considerando la versione ridotta del grafo e mantenendo come obiettivo l'apprendimento del task di link prediction sugli archi *HasDisease*.

Al termine della fase di training sono state valutate le prestazioni dei modelli utilizzando le metriche di Mean Rank (MR), Mean Reciprocal Rank (MRR) e Hits@1 (H@1).

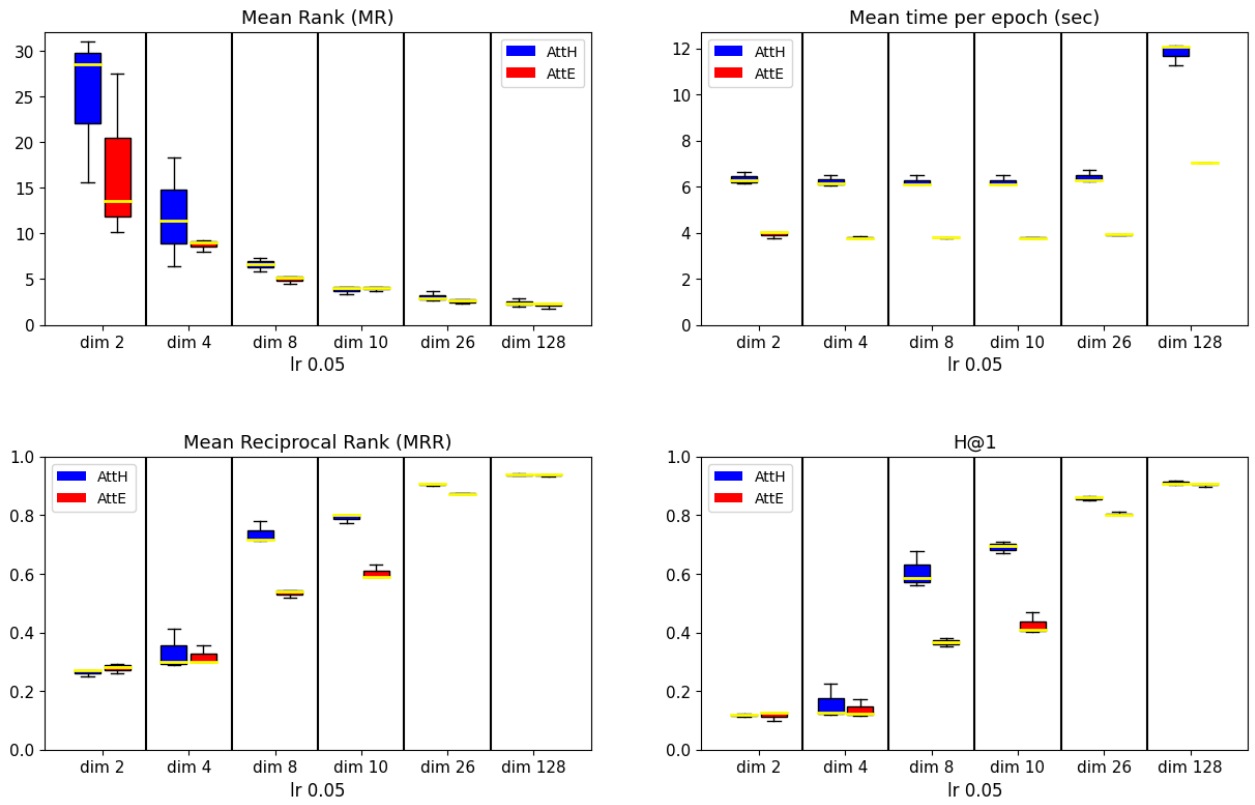


Figure 23: Comparazione modelli basati su grafi eterogenei, AttH vs AttE

La Figura 23 mostra i risultati ottenuti fissando il learning rate a 0.05 ed esplorando diverse dimensioni [2, 4, 8, 10, 26, 128]. Anche in questo caso, il training è stato condotto selezionando un numero di epoche sufficiente a raggiungere la convergenza sul validation set. In particolare, per i trial con dimensione inferiore a 128 sono state utilizzate 50 epoche, mentre per il trial con dimensione pari a 128 le epoche sono state 1 000.

Dai risultati emerge una proprietà caratteristica introdotta dall'uso degli spazi iperbolici: per dimensioni inferiori a 26, il modello AttH mostra prestazioni migliori rispetto ad AttE. Questo è dovuto alla geometria iperbolica, che garantisce maggiore rappresentazione anche a bassa dimensionalità, migliorando di conseguenza le metriche.

In maniera analoga alle differenze tra HGCN e GCN anche AttH impiega un tempo decisamente più lungo rispetto a AttE, allungando l'esecuzione del 40%, ma anche in questo caso ottenendo performance migliori nelle metriche come MRR dove guadagna un 14% medio (fino a 29% con dimensione 8) e nella metrica H@1, dove guadagna in media il 21% rispetto a AttE.

Entrambi i modelli basati su grafi eterogenei dimostrano di aver appreso in maniera eccellente il task di predizione sugli archi *HasDisease*, come mostrato dalla Figura 23, arrivando a predire in maniera accurata fino al 92% (H@1) delle diagnosi testate.

	dim 2		dim 4		dim 8		dim 10		dim 26	
Metric	AttE	AttH	AttE	AttH	AttE	AttH	AttE	AttH	AttE	AttH
MR ↓	**22.52**	22.78	**11.12**	13.06	**5.31**	7.20	**5.62**	6.49	**3.28**	4.19
MRR ↑	0.27	0.27	0.28	**0.37**	0.53	**0.75**	0.56	**0.78**	0.87	**0.88**
H@1 ↑	0.12	**0.13**	0.12	**0.22**	0.35	**0.63**	0.39	**0.68**	0.80	**0.83**
H@3 ↑	**0.34**	0.29	0.33	**0.41**	0.63	**0.84**	0.67	**0.86**	0.92	0.92
H@10 ↑	0.57	**0.60**	0.62	**0.69**	0.89	**0.93**	0.89	**0.94**	0.97	0.97

Table 5: Confronto medio su 3 seed tra AttE e AttH (in grassetto la metrica migliore)

3.6.5 Comparazione diretta tra metodi omogenei e eterogenei

Un'ulteriore analisi comparativa è stata condotta confrontando direttamente i metodi su grafi omogenei con quelli su grafi eterogenei, esaminando i vantaggi di ciascuna scelta sia dal punto di vista del piano geometrico sia in relazione alle caratteristiche del grafo (omogeneo o eterogeneo).

Per effettuare questo confronto sono stati selezionati i trial con la dimensione dell'embedding che ha mostrato le migliori performance su tutti i modelli analizzati; in questo caso, è stata scelta la dimensione pari a 26.

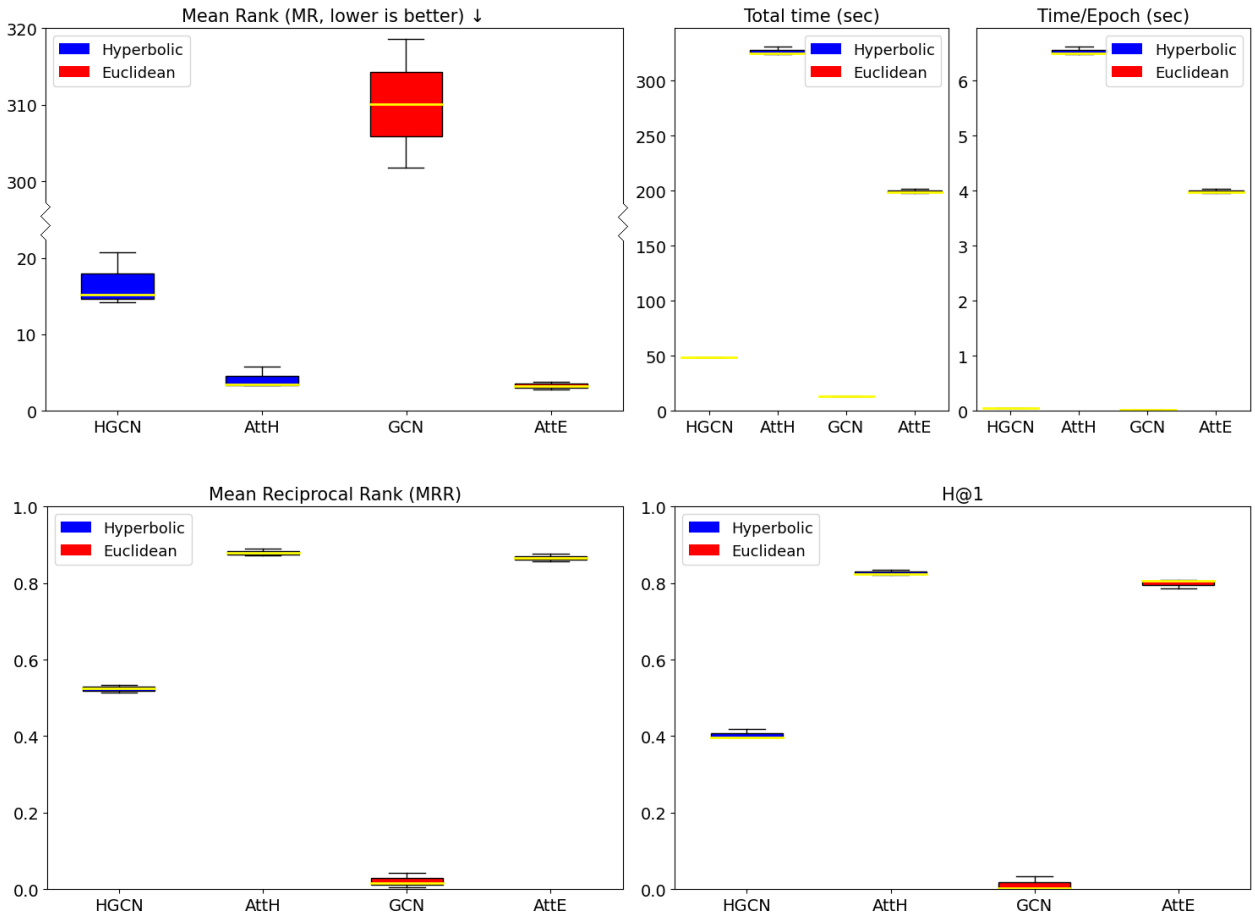


Figure 24: Confronto tra GCN, HGCN, AttE e AttH su metriche strutturali

I confronti sono stati effettuati sulle metriche strutturali che rappresentano maggliormente la capacità di apprendimento del task di predizione, confrontando anche i tempi di esecuzione per poter analizzare i costi e i vantaggi delle varie scelte.

I grafici in Figura 24 evidenziano chiaramente le differenze tra i modelli: in particolare, l'analisi dei tempi di esecuzione mostra che il passaggio da un contesto omogeneo a uno eterogeneo comporta un costo di apprendimento superiore di oltre 6 volte. Questo costo sul tempo di esecuzione è giustificato da un importante aumento della precisione delle predizioni, infatti si passa da un MRR di 0.52 a un MRR di 0.86, valori di H@1 da 0.4 a 0.8.

Model	**MR** $\downarrow$	**MRR** $\uparrow$	**H@1** $\uparrow$	**H@3** $\uparrow$	**H@10** $\uparrow$	**Time (s)**
GCN	310.11	0.022	0.012	0.018	0.021	**13.32**
HGCN	16.67	0.523	0.404	0.586	0.734	48.46
AttE	**3.28**	0.866	0.799	**0.923**	**0.970**	199.66
AttH	4.18	**0.881**	**0.827**	**0.923**	0.966	326.66

Table 6: Confronto medio finale tra modelli su diverse metriche (dimensione 26)

3.6.6 Analisi finale sul KG ridotto: inclusione dei nodi di tipo *Gene*

La sperimentazione finale della ricerca si è concentrata sullo studio delle informazioni utilizzate per il training dei modelli. I modelli, infatti, hanno basato il loro apprendimento su un grafo ridotto rispetto al grafo originale, andando ad imparare le relazioni solamente tra pazienti, malattie e fenotipi, escludendo una parte importante delle informazioni di tipo genetico.

Per valutare se le informazioni presenti siano adeguate all'apprendimento del task, è stata utilizzata una versione del grafo più estesa, sebbene ancora significativamente ridotta rispetto a quella originale, in cui vengono mantenute anche le connessioni tra i nodi *Phenotype* e *Disease* e i nodi di tipo *Gene*.

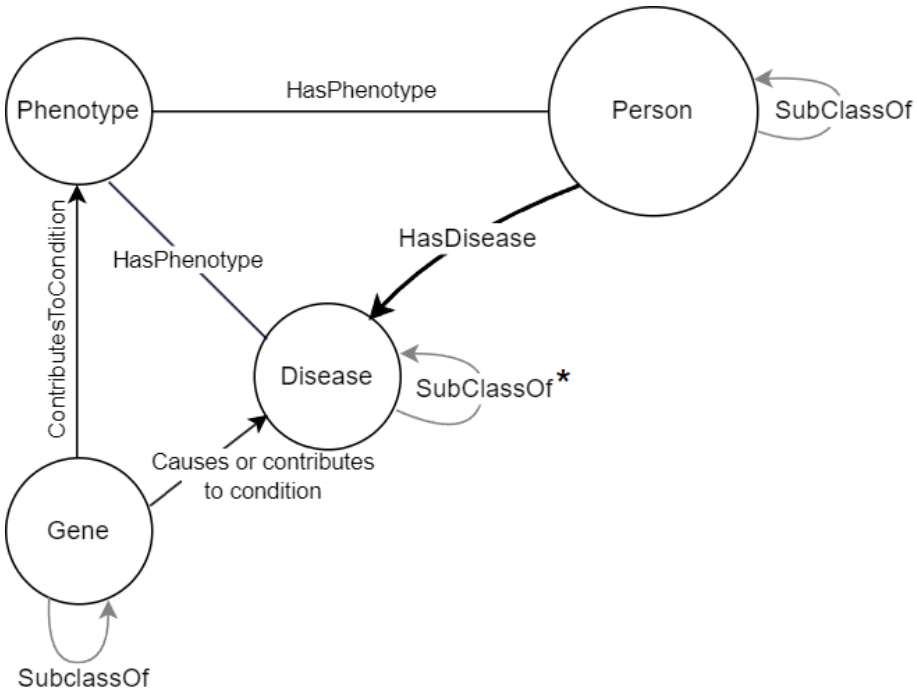


Figure 25: Grafo ridotto, ma con l'aggiunta dei nodi *Gene*

La Figura 25 mostra il nuovo grafo utilizzato: la struttura rimane identica al grafo ridotto utilizzato in precendeza, vengono solamente inclusi i nodi di tipo *Gene*, permettendo ai modelli di aggiungere contensto alle informazioni riguardanti malattie e sintomi.

Le sperimentazioni, anche in questo caso, si sono svolte in maniera analoga a quelle precendenti, sia per i modelli omogenei, sia per quelli eterogenei, mantenendo lo stesso tipo di esplorazione degli iperparametri.

La suddivisione del dataset sugli split di train, validation e test (tabella 7) è rimasta simile: il test set rimane delle dimensioni di 780 (10% degli archi totali di tipo *HasDisease*) + 780 di archi negativi, mentre tra train e validation la proporzione rimane $\approx 95/5$.

Split	**Numero di archi**	**%**
Train	197 142	94.6
Validation	9 636	4.6
Test	1 560	0.8
Totale	208 338	100

Table 7: Suddivisione del nuovo dataset in train, validation e test set

Il dataset finale è ora composto da 36 171 nodi e 208 338 archi.

I quattro modelli sono stati confrontati direttamente, valutandone le prestazioni tramite metriche strutturali. Come mostrato nella Figura 26, non emergono differenze significative rispetto ai risultati ottenuti sul grafo ridotto privo di nodi *Gene*. Le uniche variazioni rilevanti riguardano i tempi di esecuzione, che risultano inevitabilmente aumentati, insieme a un ulteriore peggioramento delle già insufficienti prestazioni del modello GCN.

Complessivamente, l'inclusione dei nodi *Gene* ha avuto un impatto negativo, introducendo criticità senza apportare miglioramenti significativi alle prestazioni. Questo dimostra che spesso l'aggiunta di informazioni superflue, oltre ad aumentare i tempi di esecuzione, ne peggiora le performance: un incremento della quantità di dati da rappresentare durante l'apprendimento può rendere più complessa la gestione dello spazio di embedding, soprattutto quando le informazioni non sono adeguatamente contestualizzate rispetto al resto dei dati.

Analizzando la scelta di comprendere nel grafo ridotto anche i soli nodi *Gene*, senza affiancarli con i nodi *Protein* causa una perdita delle informazioni fulcro delle relazioni tra geni e proteine.

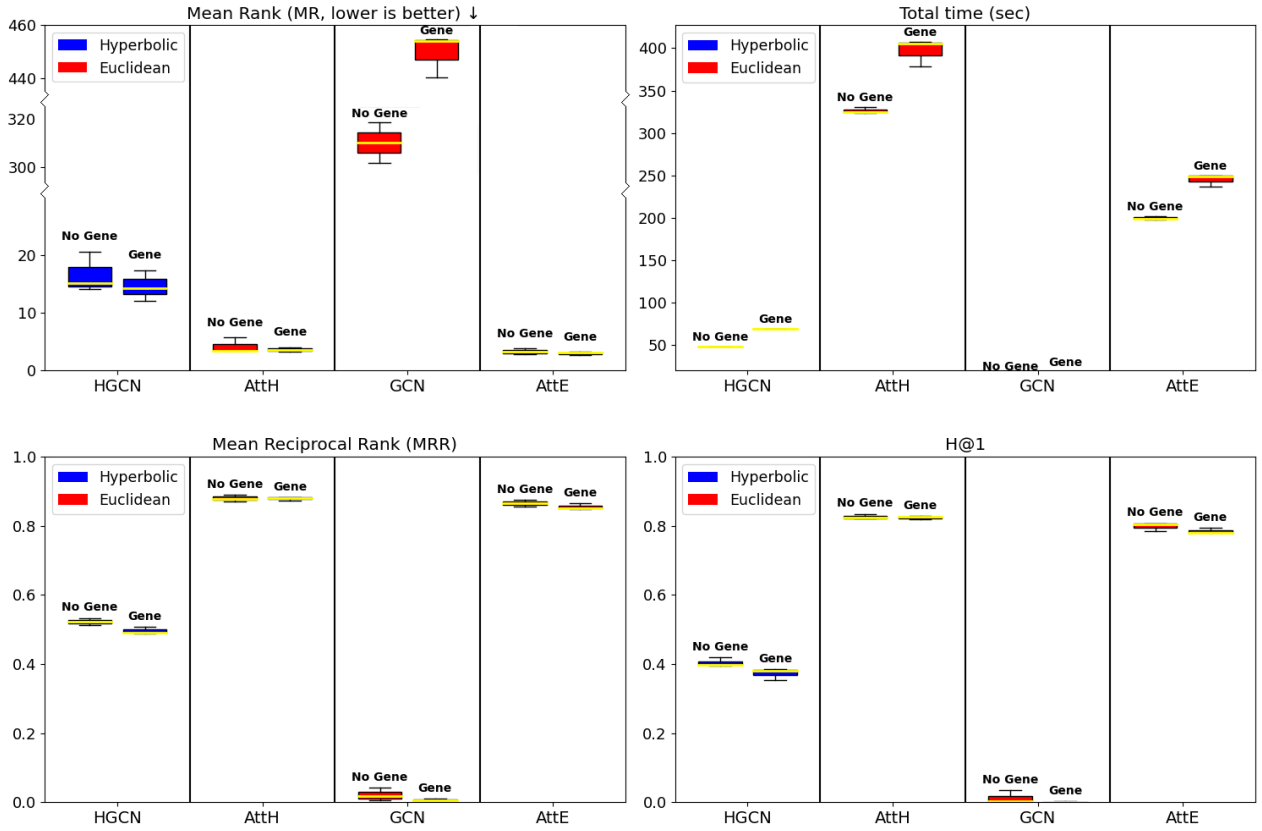


Figure 26: Confronto tra HGCN, GCN, AttH e AttE su metriche strutturali per i due diversi dataset utilizzati

L'esclusione delle proteine introduce una perdita fondamentale delle informazioni. Sebbene il gene contenga l'informazione genetica, è la proteina a costituire l'unità funzionale effettiva: la maggior parte dei meccanismi patogenetici si manifesta infatti attraverso alterazioni della struttura proteica, modifiche post-traslazionali o interazioni tra le proteine all'interno della cellula, informazioni che il solo dato genomico non può descrivere.

Come descritto in precedenza, a causa dei limiti di memoria derivati dalla libreria utilizzata non è stato possibile utilizzare il grafo interamente, anche solo l'aggiunta dei nodi *Protein* a quest'ultima versione del grafo farebbe aumentare di molto il numero di archi totali: le interazioni tra proteine, come si può osservare dalla tabella 11, compongono una rete di archi molto grande, per cui impraticabile per i limiti già descritti.

In un'ultima analisi, i modelli sono stati ordinati in base alle performance su MRR e H@1, al fine di valutare il gap reduction associato alla scelta del modello e confrontando anche la modifica strutturale con l'inserimento dei nodi *Gene*.

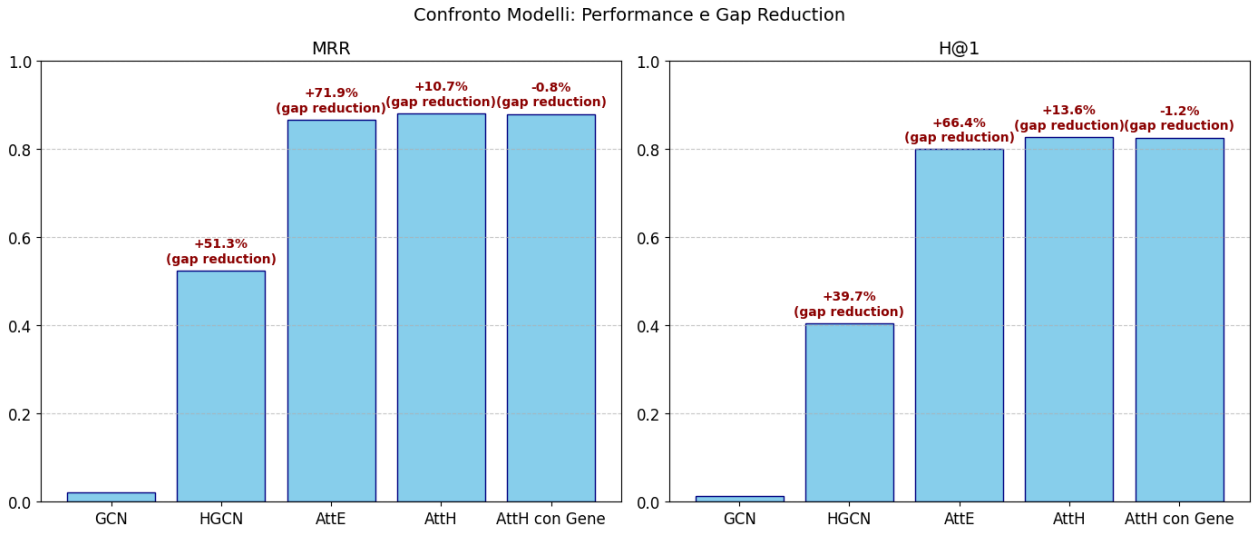


Figure 27: Guadagno in termini di gap reduction per ogni scelta di modello

Dalla Figura 27 emerge che l'adozione dello spazio iperbolico, rispetto a quello euclideo, comporta una riduzione del gap del 51.3%, mentre l'utilizzo di modelli su grafo eterogeneo, rispetto a quelli omogenei, produce la riduzione più significativa del 71.9%. Tra i modelli eterogenei, il passaggio da spazio euclideo a iperbolico porta ad un miglioramento più contenuto, pari al 10.7%.

Infine, la scelta di aggiungere solamente i nodi di tipo *Gene* al KG ridotto ha portato una perdita nelle performance, seppur lieve, del -0.8%.

4 CONCLUSIONI

La ricerca ha esplorato l'applicazione di tecniche avanzate di Graph Embedding per la rappresentazione di Knowledge Graph in ambito biomedico, con l'obiettivo primario di supportare la diagnosi delle malattie mendeliane. I risultati ottenuti confermano che la scelta dello spazio geometrico e la tipologia di modello utilizzato incidono in modo determinante sulle capacità predittive dell'intelligenza artificiale applicata a reti biologiche complesse.

In primo luogo, lo studio ha dimostrato la superiorità della geometria iperbolica rispetto a quella euclidea nel modellare dati che presentano una topologia fortemente gerarchica ad albero. I test effettuati hanno evidenziato che il modello HGCN ha ottenuto un miglioramento medio del 23% nella metrica ROC rispetto al tradizionale modello GCN nel task di link prediction sulle malattie. Questo risultato valida la capacità dello spazio iperbolico di espandersi esponenzialmente, modellando le gerarchie complesse con una distorsione minima.

In secondo luogo, il confronto tra modelli omogenei ed eterogenei ha evidenziato un salto qualitativo cruciale. Poiché i classici modelli di Graph Convolutional (come GCN e HGCN) operano su grafi omogenei, essi perdono l'informazione semantica legata ai diversi tipi di nodi e archi. L'adozione di modelli eterogenei (come AttE e AttH) ha portato a un drastico aumento della qualità delle predizioni, generando la riduzione più significativa del gap di performance, pari al 71,9%. In particolare, questi modelli sono riusciti a predire in maniera accurata fino al 92% (metrica H@1) delle diagnosi testate.

Infine, l'analisi sull'inclusione di nuovi dati ha rivelato l'importanza critica della contestualizzazione biologica. L'inserimento nel grafo di nodi di tipo *Gene* senza i corrispettivi nodi *Protein* ha causato un peggioramento delle prestazioni. Questo dimostra che il solo dato genomico isolato non è sufficiente, poiché è la proteina a costituire l'unità funzionale effettiva attraverso cui si manifestano i meccanismi patogenetici. L'aggiunta di informazioni non strutturate in modo biologicamente coerente complica lo spazio di apprendimento senza fornire reali migliorie nell'apprendimento.

4.1 Sviluppi futuri e possibili migliorie

Partendo dalle limitazioni riscontrate durante le sperimentazioni, è possibile indagare in diverse aree di miglioramento per ottimizzare l'approccio proposto:

- Ottimizzazione computazionale per l'uso dell'intero Knowledge Graph: durante la fase di training, la libreria utilizzata (hgcn) ha mostrato inefficienza nell'uso della memoria. Questa problematica tecnica ha costretto l'utilizzo di una versione fortemente ridotta del grafo, composta da soli 28 811 nodi e 178 320 archi. L'adozione o lo sviluppo di framework più ottimizzati a livello di memoria permetterebbe di addestrare i modelli sull'intero grafo finale, il quale conta 243 053 nodi e oltre 2,7 milioni di archi.

- Integrazione completa della rete di interazioni proteiche: a causa dei limiti sopracitati riguardanti la memoria, si è resa necessaria l'esclusione dei nodi *Protein*, i quali compongono una rete di archi estremamente vasta. Poiché l'assenza di questi nodi causa la perdita di informazioni fondamentali sulle alterazioni della struttura proteica e sulle interazioni cellulari, l'inclusione di questo strato informativo rappresenta la miglioria biologica più incisiva per raffinare l'accuratezza diagnostica.

- Sfruttamento delle feature dei nodi: le sperimentazioni comparative tra i modelli omogenei non hanno previsto l'uso di feature associate ai singoli nodi, basandosi esclusivamente sulla struttura topologica del grafo. L'integrazione di attributi specifici per ciascuna entità clinica o genomica all'interno delle architetture di rete neurale potrebbe incrementare ulteriormente le performance e la robustezza delle predizioni.

- Combinazione degli spazi di rappresentazione: una recente ricerca prodotta da AnacletoLab ha analizzato la combinazione di spazi euclidei e iperbolici attraverso un unico modello di KG embedding. La ricerca ha unito la rappresentazione dei modelli AttE e AttH in un unico spazio su cui si relazionano piani euclidei e iperbolici, mostrando performance molto convincenti sul KG delle malattie mendeliane.

5 BIBLIOGRAPHY

[1] Monica H. Wojcik et al. "Beyond the exome: What's next in diagnostic testing for Mendelian conditions". In: *The American Journal of Human Genetics* 110.8 (Aug. 2023), pp. 1229–1248. ISSN: 0002-9297. DOI: 10.1016/j.ajhg.2023.06.009.

[2] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. "DeepWalk: Online Learning of Social Representations". In: *CoRR* abs/1403.6652 (2014). arXiv: 1403.6652.

[3] Tomás Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. "Distributed Representations of Words and Phrases and their Compositionality". In: *CoRR* abs/1310.4546 (2013). arXiv: 1310.4546.

[4] Aditya Grover and Jure Leskovec. "node2vec: Scalable Feature Learning for Networks". In: *CoRR* abs/1607.00653 (2016). arXiv: 1607.00653.

[5] László Szirmay-Kalos, András Fridvalszky, László Szécsi, and Márton Vaitkus. "Parametric Surfaces for Elliptic and Hyperbolic Geometries". In: *Mathematics* 13 (Oct. 2025), p. 3403. DOI: 10.3390/math13213403.

[6] Octavian-Eugen Ganea, Gary Bécigneul, and Thomas Hofmann. "Hyperbolic Neural Networks". In: *CoRR* abs/1805.09112 (2018). arXiv: 1805.09112.

[7] Ines Chami, Rex Ying, Christopher Ré, and Jure Leskovec. "Hyperbolic Graph Convolutional Neural Networks". In: *CoRR* abs/1910.12933 (2019). https://github.com/HazyResearch/hgcn. arXiv: 1910.12933.

[8] Aric Hagberg, Pieter Swart, and Daniel S Chult. *Exploring network structure, dynamics, and function using NetworkX*. Tech. rep. Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2008.

[9] Radim Rehurek and Petr Sojka. "Gensim–python framework for vector space modelling". In: *NLP Centre, Faculty of Informatics, Masaryk University, Brno, Czech Republic* 3.2 (2011).

[10] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[11] Ines Chami et al. “Low-Dimensional Hyperbolic Knowledge Graph Embeddings”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. https://github.com/HazyResearch/KGEmb. 2020, pp. 6901–6914.

[12] Galileo Namata, Ben London, Lise Getoor, and Bert Huang. “Query-driven learning of visual models”. In: *Workshop on Mining and Learning with Graphs (MLG)* 8 (2012).

[13] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. “Automating the construction of internet portals with machine learning”. In: *Information Retrieval* 3.2 (2000), pp. 127–163.

[14] Università degli Studi di Milano. *AnacletoLab, Computational Biology and Bioinformatics Lab*. 2001. URL: https://anacletolab.di.unimi.it/ (visited on 03/28/2026).

[15] Payal Chandak, Kexin Huang, and Marinka Zitnik. “Building a knowledge graph to enable precision medicine”. In: *Scientific Data* 10.1 (2023), p. 67.

[16] Tiffany J. Callahan et al. *An Open-Source Knowledge Graph Ecosystem for the Life Sciences*. 2024. arXiv: 2307.05727 [cs.AI].

[17] Daniel Danis et al. “A corpus of GA4GH Phenopackets: case-level ing for genomic diagnostics and discovery”. en. In: *medRxiv* (May 2024).

[18] K. M. Tahsin Hassan Rahit and Maja Tarailo-Graovac. “Genetic Modifiers and Rare Mendelian Disease”. In: *Genes* 11.3 (2020). ISSN: 2073-4425. DOI: 10.3390/genes11030239.

[19] Aditya Rao et al. “Phenotype-driven gene prioritization for rare diseases using graph convolution on heterogeneous networks”. In: *BMC Medical Genomics* 11 (July 2018). DOI: 10.1186/s12920-018-0372-8.

6 RINGRAZIAMENTI

Il primo ringraziamento va certamente al mio relatore Mauricio e relatrice Elena che mi hanno accompagnato in questo interessante lavoro di tesi, facendomi amare sempre di più questo lavoro, come già avevano fatto Massimo e Gabriella in triennale.

Un grazie di cuore va alla mia famiglia che mi ha permesso di essere arrivato fin qua, sostenendomi fin da sempre, permettendomi di aggrapparmi a loro in tutte le mie difficoltà, riservando sempre fiducia in me e nelle mie capacità.

L'ultimo ringraziamento va infine ai miei amici, che mi hanno accompagnato nella vita, facendomi sempre sentire compreso e alleggerendo le mie giornate con spensieratezza.